



Online joint optimization of order picking process in robotic mobile fulfillment systems[☆]

Guoshuai Jiao^{a,b}, Min Huang^a, Yang Song^a, Haobin Li^c, Xingwei Wang^d

^a College of Information Science and Engineering, Northeastern University, Shenyang, Liaoning, 110819, China

^b College of Economy and Management, Taiyuan University of Science and Technology, Taiyuan, Shanxi, 030024, China

^c Department of Industrial Systems Engineering and Management, National University of Singapore, 117576, Singapore

^d College of Computer Science and Engineering, Northeastern University, Shenyang, Liaoning, 110169, China

ARTICLE INFO

Keywords:

Robotic mobile fulfillment systems
Online joint optimization
MILP model
Warehouse
Dynamic system

ABSTRACT

A robotic mobile fulfillment system (RMFS) constitutes a highly intricate and interdependent decision-making system, encompassing numerous closely related and interacting decision challenges. Meanwhile, RMFS is a prime example of a dynamic system in which various types of information, such as inventory levels, order backlogs, robot statuses, and pod locations, continuously change throughout operation. This study focuses on the online decision-making process for joint optimization of order assignment, pod selection, and task allocation within RMFS during the order-picking process. Additionally, it considers the influence of current decisions on future decisions to improve online decision-making efficiency. Initially, a dynamic system model for RMFS is developed. Subsequently, a novel mixed-integer linear programming (MILP) model is formulated to address this critical order-picking challenge effectively. Furthermore, a customized heuristic approach, employing alternating decision-making and greedy selection techniques, is proposed to overcome computational challenges associated with larger-scale RMFS scenarios. In different scales, the proposed approach outperforms a sequential approach commonly used in practice and a MILP model in existing literature for online joint optimization of order assignment and pod selection. The findings highlight the efficacy of the proposed approach in optimizing the RMFS order-picking process, particularly for larger instances, delivering significant benefits in terms of increased throughput and reduced robot travel distances.

1. Introduction

The exponential growth of e-commerce has led to an increased demand for logistics, particularly in the business-to-consumer market [1, 2]. However, market characteristics, such as stringent delivery times, extensive product assortments, and fluctuating workloads, present significant challenges to the order-picking process [3]. The order-picking process is regarded as the most time-intensive and costly operation in conventional manual systems (commonly referred to as picker-to-parts systems) [4], especially for companies managing numerous stock-keeping units (SKUs) in large warehouses. For example, JD.com, a leading e-commerce giant in China, oversees a vast inventory of over 5 million SKUs in its expansive warehouses.

The emergence of robotic mobile fulfillment system (RMFS) has enhanced warehouses efficiency and diminished labor costs for online retailers [5]. A notable example is the Kiva System used by Amazon.

Similar to the automated storage and retrieval system, RMFS is also a parts-to-picker system. Mobile robots are employed to transport pods or racks containing inventory to fixed picking workstations. This warehouse system eliminates the necessity for manual workers to navigate through the warehouse to retrieve items, as robots bring the necessary pods directly to the pickers. By automating transportation, the RMFS optimizes the picking process, eliminates the overhead incurred from the pickers' traveling time, and enables simultaneous picking from multiple workstations. This results in a significant increase in order fulfillment speed and accuracy while mitigating the labor costs associated with manual walking and searching within large warehouses [5].

1.1. Decision-making problems and literature review

In their review on RMFS, Azadeh et al. [6] classified relevant studies into three main areas: system analysis, design optimization, and

[☆] Area: Transportation and Logistics This manuscript was processed by Area Editor M. Battarra.

* Correspondence to: Northeastern University (NEU), Shenyang, Liaoning, 110004, PR China.

E-mail addresses: jiauguoshuai123@163.com (G. Jiao), mhuang@mail.neu.edu.cn (M. Huang), songyang1@ise.neu.edu.cn (Y. Song), li_haobin@nus.edu.sg (H. Li), wangxw@mail.neu.edu.cn (X. Wang).

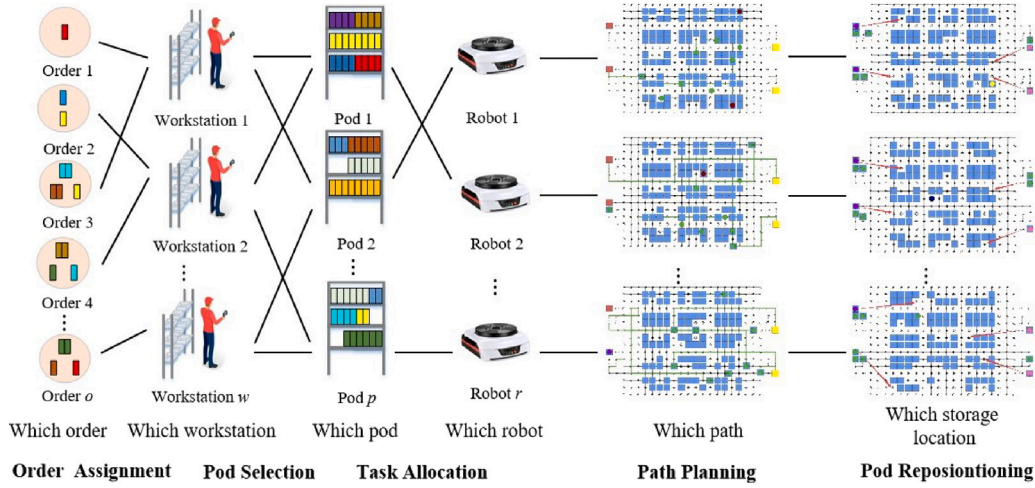


Fig. 1. Decision-making process for RMFS.

operational planning and control. In contrast, Merschformann et al. [7] and Xie et al. [8] categorized the decisions relevant to the RMFS into decisions taken at the strategic, tactical, and operational levels. This study adopts the latter classification approach. Several notable studies, such as Azadeh et al. [6], Boysen et al. [3], Merschformann et al. [7], Jaghbeer et al. [9], and da Costa Barros and Nascimento [10], provide a comprehensive literature review.

The RMFS operation process primarily comprises two procedures: the picking (retrieval or outbound) and replenishment (in short: repl., storage or inbound) processes. The picking process involves retrieving SKUs from the pods to fulfill orders, whereas the replenishment process involves putting SKUs into the pods to restock inventory. This study primarily focuses on the picking process. Therefore, we analyze picking as an illustrative example to discuss the operational level decision-making process for RMFS, as shown in Fig. 1. First, the order must be assigned to the workstation (order assignment (OA)), and, subsequently, the corresponding pod must be selected to fulfill the order (pod selection (PS)). Decisions regarding OA and PS require generating robot transportation tasks. The workstation determined by OA is the task end point and the pod determined by PS is the task starting point. The creation of these transportation tasks is followed by the allocation of the tasks to the corresponding robots (task allocation (TA)), necessitating the determination of the matching relationship between tasks and robots. Next, the robot must determine the specific traveling path (path planning (PP)) to use to execute its assigned task. After the pod completes the picking process at the workstation and returns to the storage zone, a parking location must be identified for the pod (pod repositioning (PR)). In this study, we reviewed the literature relevant to RMFS at the operational level (Table 1). Note that the operational level decision-making process for replenishment is similar to that of picking. For replenishment, the terms “OA” and “PS” in the outline above would be substituted with “ROA” and “RPS”, where ROA and RPS respectively represent replenishment order assignment and replenishment pod selection.

RMFS is a complex and interdependent decision-making system, with numerous closely interconnected decision problems. The optimal solution for a single decision may not necessarily be the global optimal solution for the entire system. Therefore, simultaneously optimizing specific decisions in RMFS is crucial for enhancing warehouse operational efficiency [48], such as OA on the demand side and PS and TA on the supply side. OA and PS aim to generate tasks for robots, whereas TA aims to allocate the generated tasks to robots. The generation and allocation of tasks are highly coupled and mutually influential. Therefore, integrating these processes for decision-making, i.e., jointly optimizing OA, PS, and TA, can effectively improve decision-making efficiency. RMFS is a typical dynamic system; inventory, backlog (i.e., all

unfulfilled orders), robot status, and pod location information changes over time during operation. Estimating this information accurately in advance constitutes a significant challenge owing to system interdependencies, hardware malfunctions, and operational delays. As highlighted by Wurman et al. [5], everything within the Kiva warehouse occurs in real time. Consequently, there is no time window during the operation of an RMFS within which operational level decision-making calculations can be made offline. Therefore, effective real time decision-making is imperative to ensure a prompt response to current system information and circumstances.

Current research on the joint optimization of OA, PS, and TA is presented in Table 2. Evidently, most existing literature focuses on offline and joint optimization of OA and PS. Zhang et al. [23] and Teck and Dewil [25,26] jointly optimized OA, PS, and TA; however, these contributions all represent offline optimization approaches. Additionally, Xie et al. [8] and Jiao et al. [28] conducted online joint optimization of OA and PS, whereas Qin et al. [22] conducted online joint optimization of PS and TA. In summary, the literature is deficient in examinations of online joint optimization of OA, PS, and TA. In this study, OA, PS, and TA in RMFS are jointly optimized online, while considering multiple workstations and inventory. Therefore, this contribution aims to address this specific literature gap regarding the requirements of real-world RMFS operational specifications.

1.2. Challenges and contributions

We address online decision-making of the joint optimization problem for OA, PS, and TA, which presents the following challenges:

- **Complexity:** the problem is intricate owing to the simultaneous consideration of multiple decision variables and constraints associated with OA, PS, and TA. Specifically, it is necessary to simultaneously determine a matching relationship among orders, pods, workstations, and robots, that is, determine which robot transports which pod to which workstation to fulfill which order.
- **Large-scale:** given the potential problem complexity, an efficient computational approach is required to handle larger-scale instances without detracting from the solution quality.
- **Real-time:** the optimization must be performed in real-time, requiring efficient and swift decision-making to meet operational requirements. Responding quickly to joint decisions in a dynamic environment is a rather intricate challenge.

Addressing the aforementioned challenges is crucial to developing an effective and practical solution for online joint optimization in RMFS. The main contributions of this study are outlined below:

Table 1
Literature review for RMFS at the operational level.

Decision problem		Publications
OA and PS	Repl.	Lamballais et al. [11] Xiang et al. [12], Weidinger and Boysen [13]; Merschformann et al. [7], Yuan et al. [14], Kim et al. [15]; Li et al. [16], Jiang et al. [17] and Zhang et al. [18]
	Pick	Wurman et al. [5], Boysen et al. [19], Merschformann et al. [7]; Jiang et al. [17], Yang et al. [20], Valle and Beasley [21]; Xie et al. [8], Qin et al. [22], Zhang et al. [23], Zhuang et al. [24]; Teck and Dewil [25,26], Wang et al. [27], Jiao et al. [28]; Justkowiak and Pesch [29,30], Subramanian and Chandrasekar [31]; Zhao et al. [32], Cheng et al. [33] and Yang et al. [34]
TA		Zou et al. [35], Yoshitake et al. [36], Gharehgozli and Zaerpour [37]; Li et al. [38], Qin et al. [22], Teck and Dewil [25,26]; Zhang et al. [23], Zhao et al. [32] and Li et al. [39]
PP		Merschformann et al. [40], Lee et al. [41], Li et al. [42]; Sun et al. [43], Luo et al. [44] and Li et al. [39]
PR		Weidinger et al. [45], Roy et al. [46], Merschformann et al. [7]; Yuan et al. [14], Li et al. [16] and Zhuang et al. [47]

Table 2
Summary of literature on the joint optimization for OA, PS, and TA.

Publications	OA	PS	TA	Online	Multiple workstations	Inventory	Approach
Boysen et al. [19]	✓	✓					MILP, Heuristic
Xie et al. [8]	✓	✓		✓	✓		MILP, Heuristic
Yang et al. [20]	✓	✓					MILP, Heuristic
Valle and Beasley [21]	✓	✓			✓	✓	MILP, Heuristic
Qin et al. [22]		✓	✓	✓	✓	✓	MILP, Heuristic
Teck and Dewil [25]	✓	✓	✓		✓		Heuristic
Teck and Dewil [26]	✓	✓	✓		✓		MILP
Zhang et al. [23]	✓	✓	✓		✓	✓	MINLP, Heuristic
Zhuang et al. [24]	✓	✓			✓		MILP, Heuristic
Wang et al. [27]	✓	✓			✓		MILP
Jiao et al. [28]	✓	✓		✓	✓	✓	MILP, Heuristic
Justkowiak and Pesch [29]	✓	✓					Heuristic
Justkowiak and Pesch [30]	✓	✓					MILP, Heuristic
Zhao et al. [32]		✓	✓		✓		MILP, Heuristic
Yang et al. [34]	✓	✓				✓	MILP, Heuristic
This study	✓	✓	✓	✓	✓	✓	MILP, Heuristic

- To the best of our knowledge, this study is the first to investigate online joint optimization for OA, PS, and TA, which is hereafter referred to as OJOOPT. Furthermore, online decision-making efficiency is improved by considering the impact of current decisions on future decisions.
- We establish a dynamic system model for RMFS and formulate a novel mixed integer linear programming model (MILP) to address OJOOPT.
- We develop a customized heuristic approach that employs alternate decision-making and greedy selection (HADGS) strategies to address large-scale instances with high efficiency.
- These inferences offer valuable managerial insights that challenge the conventional observations in the existing literature, suggesting that reducing the number of pod movements does not necessarily improve RMFS operational efficiency.

The remainder of this paper is organized as follows. Section 2 introduces a detailed problem formulation, encompassing the problem description, dynamic system modeling, and OJOOPT modeling. Section 3 describes the designed HADGS and analyzes its complexity. Section 4 presents the simulation-based numerical experiments as well as managerial insights. Finally, Section 5 summarizes this study and discusses potential future research directions.

2. Problem description and formulation

An RMFS may incorporate three types of workstations: those for replenishment, picking, and packaging. To ensure clarity and conciseness, we refer to the “picking workstations” as “workstations”, as the primary focus of this study is decision-making during the picking process.

2.1. Problem description

This study focuses on the OJOOPT in RMFS. During the operation of an RMFS, dynamic changes occur over time in many parameters such as backlog, inventory, robot status, and pod locations. Furthermore, the operation of an RMFS involves continuous human interaction (i.e., picking operations by pickers at workstations); human response time is variable and unpredictable. The consequent variability and unpredictability introduced into the system, along with dynamic system information changes necessitate real-time online decision-making for operational level related issues. Thus, the system gathers real-time information periodically in the course of its operation to make OJOOPT decisions. We introduce the variable “ t ” to represent the number of times OJOOPT is performed. Therefore, real-time system information is used as OJOOPT input at each new t .

In an RMFS, an order is typically placed in a tote at the workstation. Once all the items in the order are placed in the tote, it means that the tote is filled and subsequently transported to the packaging workstation either via a conveyor belt or by workers. Simultaneously, an empty tote is positioned in the original location. The OJOOPT decision-making process is depicted in Fig. 2. Each time t is updated, it is essential to choose multiple orders from the backlog and allocate them to their respective workstations. Subsequently, the process involves selecting appropriate pods from the available pods to fulfill assigned orders, while simultaneously selecting suitable robots from a set of available robots to transport the selected pods. The number of orders assigned in a single decision should not exceed the total available capacity of all workstations. Two scenarios exist: when the number of available robots is no less than the number of selected pods, this decision assigns

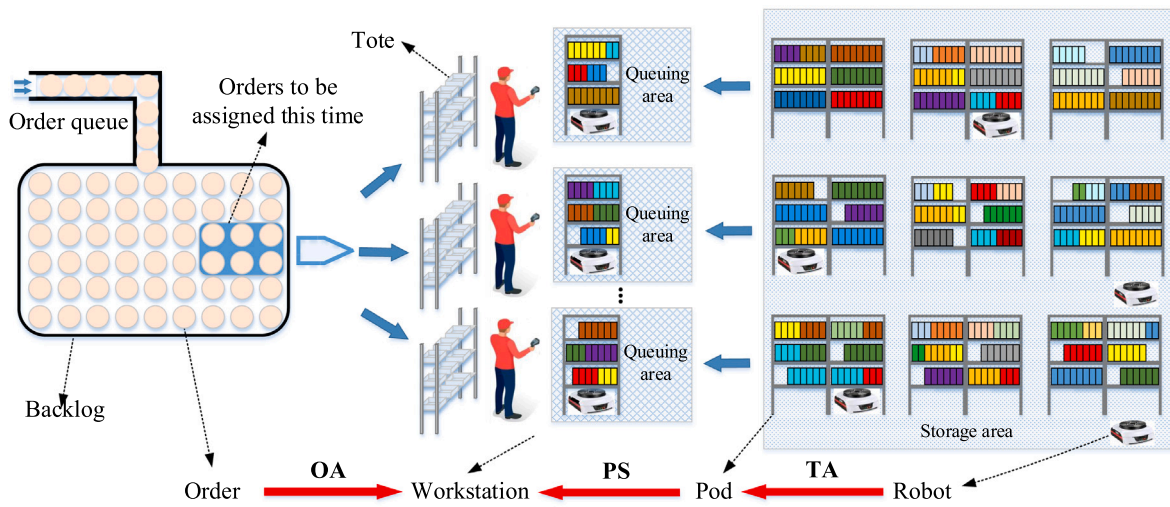


Fig. 2. OJOOPT decision-making process.

orders equivalent to the total available capacity of all workstations. In contrast, when the quantity of available robots is less than the number of selected pods, this decision assigns fewer orders than the total available capacity of all workstations. The workstation's available capacity is determined by the number of empty totes present before the decision-making process. The more empty totes there are, the higher the available capacity of the workstation. We introduce a threshold value, denoted as δ . Whenever a workstation's available capacity exceeds or equals δ , a new OJOOPT decision is triggered, increasing the value of t by 1. Therefore, we divide continuous time into a series of discrete time points by defining the OJOOPT decision as a discrete event. Specifically, for each t , we take the state of the dynamic system at the current moment as the OJOOPT input and pass the decision result to the system as the output.

Note that the pod set comprises both previously assigned pods at workstations and unassigned pods in storage zones. The former includes en-route transportation and picking up or queuing at workstations. The pods that are assigned to a workstation in the current decision can continue to be involved in future decisions as already assigned pods, as long as they do not leave the workstation. Consequently, decisions that were made online can significantly impact future decisions.

Additionally, the quantity of orders assigned to each workstation in each OJOOPT decision does not exceed its total available capacity. This implies that the quantity of orders being processed at any given time is less than or equal to the maximum processing capability of the workstation. Therefore, the necessity to sort either order sequence or pod sequence is negated. The sequence in which the pods arrive at the workstation is solely determined by their actual arrival sequence. However, in offline decision-making, the quantity of orders assigned at once frequently exceeds the processing capacity of a single workstation; therefore, order and pod sequences must be sorted. Online decision-making enables better adaptation to dynamic environmental changes, saves significant computational resources, and improves system response time.

2.2. Dynamic system modeling

We utilize RAWSim-O, a discrete-event-driven and agent-based simulation framework created by Merschformann et al. [49], to simulate a dynamic RMFS environment. By employing RAWSim-O, we create a virtual environment that simulates the dynamics and conditions of a real RMFS. This framework enables us to realistically model and simulate various system components such as orders, pods, robots, and their interactions. With RAWSim-O, we can analyze the performance

of different approaches, heuristics, or decision-making rules in a simulated RMFS environment. In other words, the relevant optimization approaches are independent of RAWSim-O. RAWSim-O enables the assessment of the effectiveness and efficiency of the proposed approaches to address the challenges and complexities of real-world RMFS scenarios. Overall, by utilizing RAWSim-O, comprehensive simulations and experiments can be conducted to provide valuable insights and guidance for optimizing RMFS decision-making processes. However, in RAWSim-O, relevant decision-making issues are optimized individually; the joint optimization approach cannot be directly applied. Therefore, to evaluate the effectiveness of the joint optimization approach, this study expands RAWSim-O, modifies its decision-making process accordingly, and uses the newly developed approach to solve OJOOPT. We name the expanded RAWSim-O "EE-RAWSim-O".

Notably, the agents in EE-RAWSim-O can be classified into two categories: one involves real entities, such as robots and workstations; the other comprises virtual entities, such as managers and solvers. All agent updates in EE-RAWSim-O are driven by discrete events. Furthermore, we utilize event graphs to depict EE-RAWSim-O modeling. Event graphs, which represent discrete event systems clearly and concisely, are graphical models. An event graph comprises event nodes connected by directed edges, which depict the cause-and-effect relationships between events. Each event node represents a specific occurrence or activity in the system, such as order arrival, task completion, or resource availability. These events are typically represented as labeled nodes in the graph. The directed edges between event nodes indicate the flow of events and the order in which they occur. For instance, an event may trigger another event to occur later, resulting in a directed edge from the initial event node to the subsequent node. Event graphs enable the visual analysis of the behavior and interactions of events in a discrete event system. The relevant parameters, events, and operators utilized in the event graph are defined in Table 3. In addition to the events defined in Table 3, such as FI, SE, FE, SP, FP, and QA, events in the event graph also include the decision events defined in Section 1, such as OJOOPT, PP, and PR. Additionally, an extraction request represents removing an SKU from the pod, that is, the picking of a single item of an order. Similarly, an insertion request means putting an SKU into the pod.

The EE-RAWSim-O event graph is shown in Fig. 3, where rectangles represent initial events, ellipses represent events other than initial events, black fonts in curly brackets represent changes in related state variables after events are triggered, and red fonts in small brackets represent trigger conditions between events. For example, when $(c_w \geq \delta)$ is true, that is, when the available capacity of workstation w is greater than or equal to the threshold δ , the OJOOPT event will

Table 3
Some notations involved in the event graph.

	Notation	Definition
Events	SI	Picker starts executing an insertion request on the repl. workstation.
	SE	Picker starts executing an extraction request on the picking workstation.
	FI	Picker has finished an insertion request.
	FE	Picker has finished an extraction request.
	PO	Generate picking orders.
	RO	Generate repl. orders.
	SP	Robot starts its movement along the pre-planned path.
	SM	Robot starts its movement within the queuing zone.
	FP	Robot finishes its movement along the pre-planned path.
	FM	Robot finishes its movement within the queuing zone.
	QA	Robot has reached the queuing zone of the workstation.
	RTA	Task allocation in the replenishment process.
	w	Index of workstations.
	p	Index of pods.
	r	Index of robots.
Parameters	W	Set of workstations.
	W^a	Set of available workstations at this time, $W^a \subseteq W$.
	P	Set of pods.
	Q	Set of transportation tasks.
	R	Set of robots.
	O	Set of orders currently in backlog.
	Q^z	Set of robots in the queuing zone.
	O^r	Set of repl. orders.
	P_w	Set of pods previously assigned to workstation w does not include those that have departed from w at the current decision, $P_w \subseteq P$.
	c_w	Current available capacity of workstation w .
	u	Maximum capacity of picking workstation.
	b	Order quantity in O during initialization.
	h_r^c	State of robot r : 0 represents idle and 1 represents busy.
	h_r^d	The type of task being executed by robot r : 0 represents picking task, 1 represents replenishment task, 2 represents repositioning task, 3 represents rest task, and 4 represents dummy task that does nothing.
	E_p^b	Set of requests for inserting into pod p .
	E_p^a	Set of requests of extraction for pod p .
	t_1	Duration required for the robot to complete a pre-planned route.
	t_2	Duration required for the robot to complete a movement in the queuing zone.
	t_3	Duration required for the picker to complete an insert request.
	t_4	Duration required for the picker to complete an extraction request.
	γ	Inventory level of the warehouse.
	ro_1	Inventory level when repl. order generation begins.
	ro_2	Inventory level when repl. order generation is stopped.
Operators	ρ	A workstation, pod, or robot serve as input, while output correspond to their respective current position.
	$-$	An decrease in parameter values or quantity of elements in the set.
	$+$	An increase in parameter values or quantity of elements in the set.
	$*$	Changes in parameter values or quantity of elements in the set, which may increase or decrease.

be triggered, which will cause the parameters in the $\{c_w-, O-, P_w+, Q*, E_p^a+, h_r^c = 1, h_r^d = 0\}$ to change. Furthermore, changes in parameters such as $c_w, O, P_w, Q, E_p^a, h_r^c$, and h_r^d can trigger other events in the event graph, enabling the entire dynamic system to operate continuously.

It should be noted that in Fig. 3, there is no one-to-one correspondence between FE and c_w . Specifically, each FE triggers a decrease in γ and E_p^a in $\{c_w+, \gamma-, E_p^a-\}$, however, this does not necessarily trigger an increase in c_w . When there are still items in an order that have not yet been picked, FE will not trigger c_w to increase. In this case, multiple FE work together to complete the picking of an order, because SE and FE form a circular structure together. The c_w increase is only triggered when all items in an order have been picked. Moreover, items in an order may be stored in multiple different pods. Therefore, when the E_p^a of a pod is 0, only PR will be triggered, and an increase in c_w will not necessarily occur.

Fig. 3 illustrates that EE-RAWSim-O is a sophisticated discrete event system, in which the occurrence of an event leads to changes in certain parameters, thereby triggering subsequent events. Note that OJOOPT solely encompasses task allocation in the picking process, while task allocation for the replenishment process must still be executed through TA. Therefore, RTA refers to the task allocation in the replenishment process.

2.3. OJOOPT modeling

First, the assumptions and notations underlying OJOOPT are outlined in Section 2.3.1. We then introduce the OJOOPT optimization objectives in Section 2.3.2. Subsequently, relevant OJOOPT constraints are detailed in Section 2.3.3. Finally, a complete MILP model of OJOOPT is presented in Section 2.3.4.

2.3.1. Assumptions and notations

The relevant OJOOPT assumptions are as follows:

Order All items in orders can be obtained in pods; thus, the warehouse's inventory can fulfill all orders. The generation of order lines and items in each order follows a normal distribution, as detailed in Appendix A.

Backlog All unfulfilled orders are in the current backlog. This study assumes that the backlog is constant, meaning that the backlog size remains unchanged during system operation. However, the proposed model and approach can also accommodate varying backlogs. When an order is assigned from the backlog, as shown in Fig. 2, an equivalent quantity of orders is added to maintain a consistent backlog size at each OJOOPT decision.

Workstation The capacity of the workstation denotes the upper limit of orders that can be processed concurrently. Typically, a workstation can process 6 to 12 orders simultaneously [8,28]. Furthermore, the time required for the picker to pick one unit of each SKU in the workstation is constant.

Pod Each pod has a predetermined capacity; thus, the quantity of SKUs that can be accommodated in each pod has a finite limit. This study adopts a shared storage policy [13], i.e., one SKU type can be accommodated in multiple pods and multiple SKU types can be accommodated in one pod. Moreover, to avoid the influence of the relative workstation position and pod delays in the queuing zone on the performance valuation of the approach, we assume that only one pod can be assigned to one workstation.

Queuing zone Each workstation corresponds to a queuing zone. As long as the workstation has one pod in the process of picking, the remaining pods arriving at the workstation must wait in the queuing zone. Note that the movement of pods in the queuing zone is not managed by the PP (i.e., path plan), albeit by a dedicated queue manager.

Table 3 provides descriptions of some notations utilized in OJOOPT, while the remaining notations are presented in Table 4. We further explain the definitions of $x_{o,w}$ and $\hat{x}_{o,w}$. $x_{o,w}$ indicates that o is assigned

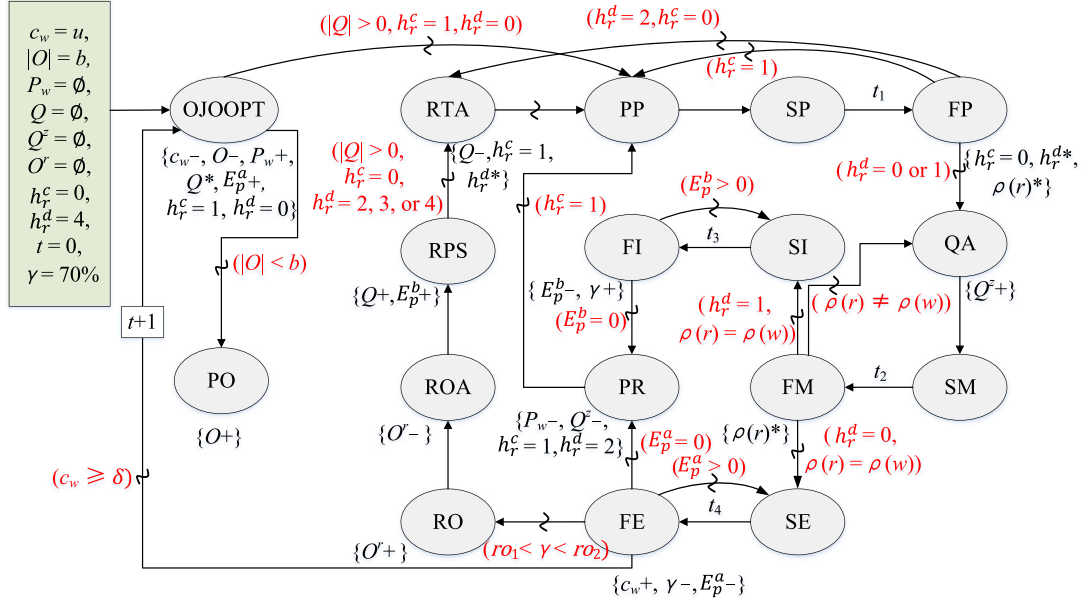


Fig. 3. EE-RAWSim-O event graph.

to w in the current decision, whereas $\hat{x}_{o,w}$ indicates that o could potentially be assigned to w in the current decision, although that may not necessarily be the case. As pods assigned to workstation w can fulfill more orders than required, the number of orders that can be assigned can exceed the actual number of orders assigned.

As previously stated, OJOOPT utilizes real-time system information as the input for each decision-making process. Hence, certain parameters in the model vary with each t . To distinguish these parameters corresponding to different t , we append the notation (t) to them. The state variable D of the dynamic system can be mathematically formulated as $D(t) = \langle R^a(t), R^b(t), W^a(t), O(t), P^a(t), P^b(t), P_w(t), \tilde{P}_i(t), \tilde{O}_i(t), \hat{O}(t), \hat{n}_{i,p}(t), c_w(t), I_p(t) \rangle$.

2.3.2. Optimization objective

We design a new optimization objective for OJOOPT. Most studies on joint optimization for OA and PS minimize the quantity of pod movements, which corresponds to reducing pod visits to workstations [8,19–21,24,27,29,30]. Furthermore, typically, the optimization objective in joint optimization for OA, PS, and TA is to minimize the overall system cost during operation [23,25,26]. This includes both the transportation cost incurred by robots and the cost associated with picking operations. The aforementioned studies on joint optimization for OA, PS, and TA primarily address offline optimization, which calculates the overall system cost to fulfill all orders in the backlog. However, in OJOOPT, a significant amount of important information changes dynamically over time, and only part of the orders in the backlog are assigned in each decision. Therefore, it is impossible to calculate the overall cost or total quantity of pod movements required by the system to fulfill all orders in the backlog in a single decision. Therefore, the new OJOOPT optimization objective is divided into two parts: long- and short-term.

Specifically, we consider maximizing the quantity of orders in the backlog that can be fulfilled on the chosen pods as a long-term objective. This involves selecting the pods with the highest capacity for order fulfillment at the time of decision-making. The selected pods in the current decision fulfill more orders in the backlog, resulting in fewer pods being chosen for future decisions to minimize the total pod movement required during long-term operation. Furthermore, we consider minimizing the robot traveling distance required to fulfill the orders assigned by the current decision as a short-term objective, i.e., selecting the pod and robot with the shortest robot travel distance.

Table 4
Some notations in OJOOPT.

	Notation	Definition
Indices	i	Index of SKU.
	o	Index of order.
	p^r	Index of pod transported by robot r .
Parameters	P^a	Set of unallocated pods still in storage zones, $P^a \subseteq P$.
	P^b	Set of previously allocated pods that have not completed the extraction task, $\sum_{w \in W} P_w = P^b$ and $P^b \subseteq P$.
	I	Set of SKU types within the warehouse.
	I_p	Set of SKUs accommodated in pod p , $I_p \subseteq I$.
	I_o	Set of SKUs contained in order o , $I_o \subseteq I$.
	$\tilde{O}_i$	Set of orders containing SKU i , $\tilde{O}_i \subseteq O$.
	$\tilde{P}_i$	Set of pods that accommodate SKU i , $\tilde{P}_i \subseteq P$.
	R^a	Set of idle robots, $R^a \subseteq R$.
	R^b	Set of busy robots, $R^b \subseteq R$.
	$\hat{n}_{i,p}$	Quantity of SKU i accommodated in pod p .
	$n_{i,o}$	Quantity of SKU i contained in order o .
	τ	Threshold for residence time of order.
	g_o	Residence time of order o in O .
	$\hat{O}$	Set of orders that exceed the time threshold, $\hat{O} \subseteq O$.
	$d_{p,w}, d_{r,p}$	Distance between pod p and workstation w and between the current position of robot r and pod p .
Variables	$x_{o,w}$	Binary variable: 1, if order o is assigned to workstation w .
	$\hat{x}_{o,w}$	Binary variable: 1, if order o can be assigned to workstation w .
	$y_{r,p}$	Binary variable: 1, if robot r is allocated to pod p .
	$z_{p,w}$	Binary variable: 1, if pod p is allocated to workstation w .
	$\delta_{o,p,w}$	Binary variable: 1, if some SKUs in the assigned order o are supplied by pod p assigned to workstation w .
	u_w	Integer variable: unused capacity of workstation w .

The robot traveling distance comprises two components: pick-up trip (from robot to pod position) and forward trip (from pod to workstation position). Therefore, the robot traveling distance is determined by both the selected pod and robot. The shorter the robot travel distance in a single decision, the less likely they are to travel a lengthy total distance over multiple decisions. For example, in a certain decision, we choose between selecting a pod that is close but fulfills fewer orders or selecting a pod that is far but fulfills more orders. Therefore, short-term objectives focus more on the current decision, while long-term objectives focus on the effect of the current decision on future decisions. Therefore, we must balance the impact of long- and short-term objectives on the system. The OJOOPT optimization objective is expressed as follows:

$$\min \alpha_1 \times f_1(t) + \alpha_2 \times f_2(t) \quad (1)$$

where f_1 and f_2 represent long- and short-term objectives, respectively, and α_1 and α_2 represent the weights of f_1 and f_2 . The impact of long- and short-term objectives on the system can be balanced by adjusting the size of α_1 and α_2 , thereby maximizing system operational efficiency.

The newly designed optimization objective guides the current decision through a short-term objective and ensures that the current decision in OJOOPT has a positive influence on subsequent decisions through a long-term objective, although such influence cannot be accurately quantified.

2.3.3. Constraints

In OJOOPT, there are several constraints that must be satisfied. These constraints can be divided into four categories, as follows:

(1) Constraints between orders and workstations

$$\sum_{w \in W^a(t)} \hat{x}_{o,w} \leq 1, \forall o \in O(t) \quad (2)$$

$$x_{o,w} \leq \hat{x}_{o,w}, \forall o \in O(t), w \in W^a(t) \quad (3)$$

$$\sum_{o \in O(t)} x_{o,w} = c_w(t) - u_w, \forall w \in W^a(t) \quad (4)$$

Constraint (2) requires that one order be assigned to one workstation at most. Constraint (3) means that orders that can be assigned to workstations may not necessarily be assigned during this decision, and orders that are assigned to workstations during this decision must be assigned. Constraint (4) indicates that the quantity of orders assigned to workstation w is equivalent to w 's available capacity minus its unused capacity.

(2) Constraints between pods and workstations

$$\sum_{o \in \hat{O}_i(t)} n_{i,o} \cdot \hat{x}_{o,w} \leq \sum_{p \in \hat{P}_i(t)} \hat{n}_{i,p}(t) \cdot z_{p,w}, \forall i \in I, w \in W^a(t) \quad (5)$$

$$\sum_{w \in W^a(t)} z_{p,w} \leq 1, \forall p \in P \quad (6)$$

$$z_{p,w} = 1, \forall w \in W^a(t), p \in P_w(t) \quad (7)$$

Constraint (5) ensures that demand does not exceed supply, meaning that the quantity of SKU i required for all orders assigned to workstation w cannot exceed the quantity of SKU i that can be provided by all pods assigned to w . Constraint (6) guarantees that a pod can only be assigned to a single workstation at most. Constraint (7) guarantees all pods in P_w can be assigned to workstation w , indicating that the current decision inherits the pods previously assigned to w albeit have not yet left w .

(3) Constraints between robots and pods

$$\sum_{w \in W^a(t)} z_{p,w} \leq \sum_{r \in R} y_{r,p}, \forall p \in P \quad (8)$$

$$\sum_{r \in R} y_{r,p} \leq 1, \forall p \in P \quad (9)$$

$$\sum_{p \in P} y_{r,p} \leq 1, \forall r \in R \quad (10)$$

$$y_{r,p^r} = 1, \forall r \in R^b(t) \quad (11)$$

Constraint (8) ensures that when a pod is assigned to a workstation, there must be a corresponding robot to transport the pod. Constraint (9) ensures that one pod can only be transported by one robot at most. Constraint (10) means that one robot can transport up to one pod only. Constraint (11) ensures that the occupied robot continues to perform the current transport task.

(4) Constraints between orders, pods, and workstations

$$2 \cdot \delta_{o,p,w} \leq x_{o,w} + z_{p,w}, \forall w \in W^a(t), p \in P^a(t), o \in O(t) \quad (12)$$

$$z_{p,w} \leq \sum_{o \in O(t)} \delta_{o,p,w}, \forall w \in W^a(t), p \in P^a(t) \quad (13)$$

Constraint (12) ensures that when order o is supplied by pod p at workstation w , o and p must be assigned to w . Constraint (13) ensures that at least one assigned order can be supplied in the newly assigned pod. It should be noted that $\delta_{o,p,w}$ is an auxiliary decision variable, and constraints (12)–(13) are all related to $\delta_{o,p,w}$. The introduction of $\delta_{o,p,w}$ and its associated constraints in our model aims to ensure that one newly assigned pod supplies at least one newly assigned order.

2.3.4. MILP model

We formulate OJOOPT, considering multiple workstations and inventory, as an MILP model referred to as M1, as follows:

$$\begin{aligned} M1(t) : \min & \alpha_1 \sum_{o \in O(t)} \sum_{w \in W^a(t)} \hat{x}_{o,w} + \alpha_2 \left(\sum_{p \in P^a(t)} \sum_{w \in W^a(t)} d_{p,w} \cdot z_{p,w} \right. \\ & \left. + \sum_{r \in R^a(t)} \sum_{p \in P^a(t)} d_{r,p} \cdot y_{r,p} \right) + \alpha_3 \sum_{w \in W^a(t)} u_w \end{aligned} \quad (14)$$

subject to: Constraints (2)–(13)

$$x_{o,w}, \hat{x}_{o,w}, z_{p,w}, \delta_{o,p,w}, y_{r,p} \in \{0, 1\}, \forall w \in W^a(t), p \in P, o \in O(t), r \in R \quad (15)$$

$$u_w \in \mathbb{Z}_0^+, \forall w \in W^a(t) \quad (16)$$

Objective (14) is an expanded version of objective (1), where $f_1(t) = \sum_{o \in O(t)} \sum_{w \in W^a(t)} \hat{x}_{o,w}$ and $f_2(t) = \sum_{o \in O(t)} \sum_{p \in P} \sum_{w \in W^a(t)} d_{p,w} \cdot \delta_{o,p,w} + \sum_{r \in R^a(t)} \sum_{p \in P^a(t)} d_{r,p} \cdot y_{r,p}$. Objective (14) has an additional penalty for workstation unused capacity by comparison to objective (1), aiming to increase $f_1(t)$ and decrease $f_2(t)$ while minimizing the amount of unused workstation capacity. Therefore, α_3 is given a larger value to reduce u_w insofar as possible. Constraints Eqs. (15) and (16) define the feasible region for the decision variables, where Eq. (15) ensures that the relevant variables are binary values and Eq. (16) ensures that u_w is a non-negative integer value. It should be noted that M1 is generalizable as it can easily consider other objectives. For example, if the objective is to minimize the number of pod movements as is commonly the case in previous studies, formula (14) can be replaced with formula (17). If the objective is to minimize robot travel distance, formula (14) can be replaced with formula (18).

$$\min \alpha_1 \sum_{p \in P^a(t)} \sum_{w \in W^a(t)} z_{p,w} + \alpha_3 \sum_{w \in W^a(t)} u_w \quad (17)$$

$$\min \alpha_2 \left(\sum_{p \in P^a(t)} \sum_{w \in W^a(t)} d_{p,w} \cdot z_{p,w} + \sum_{r \in R^a(t)} \sum_{p \in P^a(t)} d_{r,p} \cdot y_{r,p} \right) + \alpha_3 \sum_{w \in W^a(t)} u_w \quad (18)$$

Furthermore, before calling M1(t), preprocessing must be performed, which is achieved using formula (19). This means replacing O with $\hat{O}$ when the quantity of orders in $\hat{O}$ exceeds 0. This is because the proposed model consistently assigns orders with smaller objective values at each time t , which may lead to certain orders in O not being assigned for an extended period, resulting in delays. To avoid this situation, we append all orders with a residence time greater than τ to $\hat{O}$ at each t ; thus, orders in $\hat{O}$ can be assigned preferentially in OJOOPT.

$$O(t) \leftarrow \hat{O}(t), \text{ if } |\hat{O}(t)| > 0 \quad (19)$$

The transformation of the state variable $D(t)$ to $D(t+1)$ in a dynamic system can be represented by the function F , as follows:

$$D(t+1) = F(D(t), S(t), \Gamma(t)) \quad (20)$$

where $S(t) = \langle z_{p,w}, y_{r,p}, x_{o,w}, \hat{x}_{o,w}, \delta_{o,p,w}, u_w \rangle(t)$ denotes the OJOOPT decision at t and $\Gamma(t)$ denotes a stochastic factor that arises between t and $t+1$. The relationship between $D(t)$ and $D(t+1)$ is difficult to express explicitly using mathematical formulas because of the presence of $\Gamma(t)$. Specifically, $\Gamma(t)$ primarily denotes the variations in pertinent parameters in OJOOPT induced by external events in the dynamic system, as shown in Fig. 3. For example, PR causes a change in P_w , FE causes a change in c_w , and FP causes a change in h_r^c .

Moreover, $d_{r,p}$ denotes the distance traveled by robot r from its current position to the position of pod p , whereas $d_{p,w}$ denotes the distance traveled by the robot from the position of pod p to workstation w . In the RMFS map, the driving channels are directional, whereas pod storage zones are undirected. Therefore, the robot traveling distance cannot be simply expressed as the Manhattan distance. We utilize the classic A* method [50] to pre-calculate and store the distance between any two points on the map in a matrix. Therefore, the values of $d_{r,p}$ and $d_{p,w}$ can be obtained directly from this pre-calculated matrix when necessary.

The OJOOPT further considers TA based on online joint optimization for OA and PS. Jiao et al. [28] has proven that the online joint optimization problem for OA and PS is NP-hard. Thus, OJOOPT in this study is also NP-hard.

3. Heuristic

For small-scale instances, we use a commercial solver, such as Gurobi or Cplex, to solve M1. However, this exact approach does not effectively solve large-scale instances. Therefore, a heuristic approach based on alternate decision-making and greedy selection (i.e., HADGS) is designed. We embed TA into PS (denoted as PS&TA), which means selecting the nearest robot while selecting the pod. The alternate decision refers to an alternate decision between OA and PS&TA. Greedy selection refers to selecting orders, pods, and robots greedily in the OA and PS&TA, and considering each other's influence in the selection process. Therefore, HADGS realizes joint optimization of OA, PS, and TA through the combined effect of alternate decisions and greedy selection. We introduce the overall HADGS approach in Section 3.1, and the OA and PS&TA approaches in Sections 3.2 and 3.3, respectively.

3.1. HADGS

The pseudocode for the complete HADGS approach is shown in Algorithm 1. **Step 1** represents preprocessing (Lines 2–9), specifically designed to facilitate the timely assignment of all orders in O . **Step 2** initiates joint optimization. For each workstation w , order assignment is first performed by invoking the OA approach, which allows only orders that can be completely fulfilled by the pods in the P_w (i.e., the set of pods previously assigned to workstation w) to be assigned to workstation w . The term “completely fulfilled” refers to the case in which the pods in P_w can completely fulfill all items in one order, without requiring the assignment of additional pods. If no such order exists and $c_w > 0$ at this time, the PS&TA approach is invoked for pod selection and task assignment (Line 14). This only allows the pod and its corresponding robot that can completely fulfill at least one order in O to be assigned to workstation w ; the OA approach is subsequently reinvoked (Line 15). The aforementioned process is repeated until $c_w = 0$; thus, it is not necessary to continue calling the PS&TA approach, which suggests that order requirements of w have been satisfied.

In summary, the proposed HADGS approach has the following advantages:

Algorithm 1 Pseudocode for HADGS.

Input: $W^a, O, c_w, P_w, P^a, R^a$
Initialize: $\hat{O} = \emptyset$;
1: Step 1: Preprocessing
2: for (each order $o \in O$) **do**
3: if ($g_o > \tau$) **then**
4: add o to $\hat{O}$;
5: end if
6: end for
7: if ($|\hat{O}| > 0$) **then**
8: $O \leftarrow \hat{O}$;
9: end if
10: Step 2: Joint optimization
11: for (each workstation $w \in W^a$) **do**
12: OA(w, O, c_w, P_w);
13: if ($c_w > 0$) **then**
14: PS&TA(w, O, c_w, P_w, P^a, R^a);
15: goto 12;
16: end if
17: end for

- OA, PS, and TA joint optimization is realized by alternating decisions and considering each other's influence in their respective decisions. Specifically, HADGS makes the alternate decision of OA and PS-TA through cyclic calls to the OA and PS&TA approaches. In the proposed OA approach, the assigned order must be completely fulfilled by the set of pods (i.e., P_w) that have been previously assigned to the workstation. In the proposed PS&TA approach, decisions are made simultaneously for both PS and TA to ensure that the resulting P_w can completely fulfill at least one order in O .
- The greedy selection strategy is employed to consider the consequences of future decisions during the current decision-making process. In the proposed OA approach, we prioritize orders that pods close to the workstation can completely fulfill. In the proposed PS&TA approach, priority is afforded to those (robot, pod) pairs with the lowest evaluation value. Evaluation formulae (23) and (24) are concrete forms of the formula (1), as described in Section 2.3.2, where the long-term objective focuses on the consequences of current decisions on future decisions.
- The computational complexity of HADGS is $o(|W| \cdot |O| \cdot |P^a|(|P^b| + |R^a|))$; thus, HADGS is a polynomial-time approach. We transform the joint decision for multiple problems into an alternate decision for a single problem, effectively reducing the solution complexity. To avoid selecting multiple pods from P^a in turn, we incorporate a hybrid approach combining a local search and mathematical model (LSMM) into the PS&TA approach to further reduce the solution complexity.

3.2. OA approach

In the proposed OA approach, the orders assigned to workstation w must be completely fulfilled by the set of pods (i.e., P_w) that have been previously assigned to the workstation.

Next, we define the two operators, J and G , in the OA approach in Definitions 1 and 2, respectively. J represents an operator used to determine whether an order can be completely fulfilled by the pods in P_w , and G is an operator used to evaluate the order.

Definition 1. $J(o)$: The input is order o , and if formula (21) holds, the output is true; otherwise, the output is false.

$$\sum_{p \in P_w} \hat{n}_{i,p} \geq n_{i,o}, \quad \forall i \in I_o \quad (21)$$

Definition 2. $G(o, w)$: The inputs are order o and workstation w . First, the pods in P_w are sorted in ascending order of the distance between the pods and workstations. Each pod is then retrieved sequentially based on this sorted order, and pods capable of supplying items in o are added to $\hat{P}$ until o is completely fulfilled, that is, the pods in $\hat{P}$ can completely fulfill o . Finally, the sum of the distances between all pods in $\hat{P}$ and workstation w is taken as the output as follows:

$$G(o, w) = \sum_{p \in \hat{P}} d_{p,w} \quad (22)$$

Algorithm 2 provides the pseudocode for the OA approach. Specifically, for each order in O , operator J is first used to determine whether the pods in P_w can completely fulfill o . Operator G is subsequently used to evaluate the order, and finally, the order with the lowest evaluation value is assigned to the workstation. If no order exists that can be completely fulfilled by the pods in P_w , the algorithm terminates (Line 12). The stock level of the pods in the “for loop” (Lines 2–8) remains unchanged, because $J(o)$ in the “for loop” is only used to determine whether the pods in P_w can completely fulfill o . After assigning orders to w (Line 10), the stock level updates such that the corresponding stock lock in the pods used to fulfill o can no longer participate in subsequent decisions. Therefore, the stock level of the pods changes in the “while loop” (Lines 1–14). Furthermore, the complexity of the proposed OA approach is $O(c_w \cdot |O| \cdot |P_w|)$.

Algorithm 2 Pseudocode for OA approach.

Input: w, O, c_w, P_w
Initialize: $\dot{o} = \emptyset; \dot{d} = \infty$
1: **while** ($c_w > 0$ && $|O| > 0$) **do**
2: **for** (each order $o \in O$) **do**
3: **if** ($J(o)$) **then**
4: **if** ($G(o, w) < \dot{d}$) **then**
5: $\dot{d} \leftarrow G(o, w); \dot{o} \leftarrow o$;
6: **end if**
7: **end if**
8: **end for**
9: **if** ($\dot{o} \neq \emptyset$) **then**
10: assign $\dot{o}$ to w ; update stock; remove $\dot{o}$ from O ; $c_w \leftarrow c_w - 1$;
11: **else**
12: Break;
13: **end if**
14: **end while**

3.3. PS&TA approach

After OA is executed, if no order exists that can be completely fulfilled by the pods in P_w and c_w remains greater than 0 at this time, PS&TA should be executed. We embed TA into PS, i.e., select a robot closest to the pod while selecting the pod. The concept of the proposed PS&TA approach is to select one pod from P^a and add it to P_w , which can completely fulfill at least one order in O . If no such pod exists, which may require two, three or even more pods to completely fulfill an order in O , a LSMM approach is used for PS&TA.

Subsequently, we provide a function $K(p)$ to evaluate pods in PS&TA, as follows:

$$K(p) = \alpha_1 \cdot \ddot{n} + \alpha_2(d_{r,p} + d_{p,w}) \quad (23)$$

where $\ddot{n}$ represents the maximum quantity of orders that P_w can completely fulfill after adding p , which is then approximated via lines 4–8 in Algorithm 3 for the purpose of reducing runtime. Notably, formula (23) can be considered a specific version of formula (1). Therefore, formula (23) follows the weight coefficients α_1 and α_2 of formula (1).

The pseudocode for the PS&TA method is shown in Algorithm 3. For each pod p in P^a , $\ddot{n}$ is first approximated through Lines 4–8. The

robot closest to p is subsequently identified (Line 9), and p is evaluated by function $K(p)$. Finally, the $\dot{p}$ with the lowest evaluation value is obtained (Line 11). If $\dot{p}$ is an empty set, the quantity of pods that completely fulfill at least one order is greater than 1. When no single pod can fulfill at least one order, it is necessary to gradually increase the number of pods, that is, selecting two pods from P^a at a time to continue repeating the above process (Lines 2–14), which will bring huge computational burden. Moreover, when no two pods can fulfill at least one order, we must select three pods from P^a at a time, which further increases the amount of calculation required. Similarly, increasing pod quantity will significantly increase the calculation amount. Therefore, we design the LSMM approach to continue solving PS&TA.

Algorithm 3 Pseudocode for PS&TA approach.

Input: w, O, c_w, P_w, P^a, R^a
Initialize: $\dot{p} = \emptyset; \dot{r} = \emptyset; \ddot{n} = \infty$
1: **if** ($c_w > 0$ && $|O| > 0$ && $|R^a| > 0$) **then**
2: **for** (each pod $p \in P^a$) **do**
3: add p to P_w ; $\ddot{n} = 0$;
4: **for** (each order $o \in O$) **do**
5: **if** ($J(o)$) **then**
6: $\ddot{n} \leftarrow \ddot{n} + 1$; update stock;
7: **end if**
8: **end for**
9: $\dot{r} = \arg \min_{r \in R^a} (d_{r,p})$;
10: **if** ($K(p) < \ddot{n}$) **then**
11: $\ddot{n} \leftarrow K(p); \dot{p} \leftarrow p$;
12: **end if**
13: remove p from P_w ; update stock;
14: **end for**
15: **if** ($\dot{p} \equiv \emptyset$) **then**
16: LSMM(w, O, c_w, P_w, P^a, R^a);
17: **else**
18: assign $\dot{p}$ to w ; assign $\dot{r}$ to $\dot{p}$; add $\dot{p}$ to P_w ;
19: remove $\dot{p}$ from P^a ; remove $\dot{r}$ from R^a ;
20: **end if**
21: **end if**

In Algorithm 3, for each pod p , the “for loop” (Lines 4–8) is used to calculate the number of orders that P_w can completely fulfill after adding p . Therefore, after it is determined that P_w can completely fulfill order o (i.e., $J(o) = \text{true}$), the stock of corresponding pods in P_w must be reduced at this time (Line 6). However, the aforementioned stock changes only occur during the calculation process; the actual stock changes occur only after the order is assigned to the workstation (for example, Line 10 in Algorithm 2). Therefore, after removing this p from P_w , the stock must be restored to the original level (Line 13) to calculate the number of orders that the P_w can completely fulfill after adding another p .

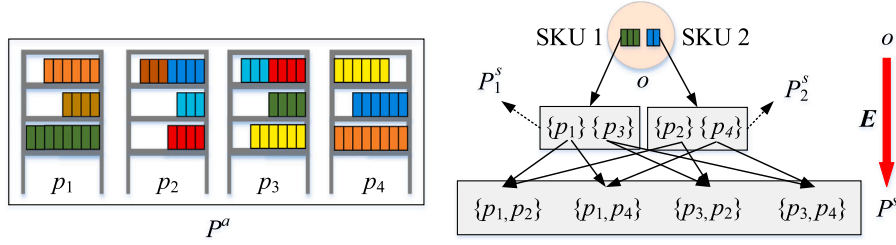
Before presenting the LSMM approach, we first introduce two notations, one operator, and one evaluation function.

P_i^s Set where each element is a set of pods that can completely fulfill SKU i .

P^s Set where each element is a set of pods that can completely fulfill order o . The elements in P^s are composed of elements from P_i^s .

Definition 3. $E(o)$: The input and output are o and P^s , respectively. The operator first obtains the corresponding P_i^s for all SKU i in o , takes one element from each P_i^s to form P^s , and ensures that the quantity of pods in each element of P^s does not exceed the quantity of idle robots in R^a .

$$KS(RP) = \alpha_1 \cdot \ddot{n} + \alpha_2 \sum_{(r,p) \in RP} (d_{r,p} + d_{p,w}) \quad (24)$$

Fig. 4. Example of P_i^s , P^s , and $E(o)$.

where RP represents the set of tuple (r, p) and tuple (r, p) represents the matching relationship between pod p and robot r (i.e., p is transported by r). Formula (24) can be considered a generalized version of formula (23). This is because formula (24) applies to the case of multiple robots and multiple pods, while formula (23) only applies to the case of one robot and one pod. Therefore, formula (24) also follows the weight coefficients α_1 and α_2 of formula (1). Moreover, if other objectives are to be considered, we can modify formulae (23) and (24) according to the modification of formula (14).

Fig. 4 shows a simple example of P_i^s , P^s , and $E(o)$. Among these, P^a has four pods: p_1 , p_2 , p_3 , and p_4 . Order o contains three SKU 1 and two SKU 2. Two sets of pods completely fulfill SKU 1: $P_1^s = \{\{p_1\}, \{p_3\}\}$. Two sets of pods completely fulfill SKU 2: $P_2^s = \{\{p_2\}, \{p_4\}\}$. Then, one element in P_i^s is taken in turn and combined to form P^s : $P^s = \{\{p_1, p_2\}, \{p_1, p_4\}, \{p_3, p_2\}, \{p_3, p_4\}\}$. Therefore, each element in P_i^s can completely fulfill SKU i contained in o , whereas each element in P^s can also completely fulfill o .

LSMM is a hybrid approach that combines local search and mathematical models, and its pseudocode is presented in Algorithm 4. First, one order o is randomly extracted from O , and P^s is obtained through $E(o)$. If P^s is an empty set, the number of pods in all elements of P^s is greater than the number of idle robots in R^a , and the algorithm returns directly. To improve computational efficiency, we do not evaluate all pod sets in P^s , albeit randomly select one pod set from P^s each time and limit the evaluation times by setting a threshold value l_1 . We subsequently utilize the 0–1 integer programming (IP) model to solve the matching relationship (i.e., RP) between pods in $\tilde{P}$ and robots in R^a . The RP is evaluated by the evaluation function KS (formula (24)) such that the RP with the minimum evaluation value can be obtained. It should be noted that the manner in which $\tilde{n}$ is obtained in formula (23) is similar to that in formula (24), and can be referred to in Lines 4–8 of Algorithm 3. Moreover, owing to the randomness in selecting the order (Line 2) and the corresponding pod set (Line 8), a local search strategy is incorporated to enhance the quality of the solution. As shown in Lines 15–18, the algorithm returns to Line 1 to repeat the aforementioned process when $i_2 < l_2$, where l_2 represents the count of the local search performed.

IP is an assignment model suitable for scenarios where the quantity of robots is not less than that of the pods. The inputs of IP are $\tilde{P}$ and R^a , and the output is the matching relationship RP between $\tilde{P}$ and R^a , as follows:

$$\text{IP : min } \sum_{r \in R^a} \sum_{p \in \tilde{P}} d_{r,p} \cdot y_{r,p} \quad (25)$$

$$\text{s.t. } \sum_{r \in R^a} y_{r,p} = 1, \forall p \in \tilde{P} \quad (26)$$

$$\sum_{p \in \tilde{P}} y_{r,p} \leq 1, \forall r \in R^a \quad (27)$$

$$y_{r,p} \in \{0, 1\}, \forall r \in R^a, p \in \tilde{P} \quad (28)$$

Objective (25) minimizes the sum of distances between robots and pods. Constraint (26) ensures that each pod is transported by a robot. Constraint (27) ensures that each robot can only transport a maximum of one pod. Constraint (28) defines the domain of the decision variable.

Algorithm 4 Pseudocode for LSMM approach.

Input: w, O, c_w, P_w, P^a, R^a ;
Initialize: $P^s = \emptyset$; $RP = \emptyset$; $\tilde{RP} = \emptyset$; $\tilde{n} = \infty$; $i_2 = 0$

- 1: **if** ($c_w > 0$ && $|O| > 0$ && $|R^a| > 0$) **then**
- 2: randomly take an order o from O ;
- 3: $P^s = E(o)$; $i_1 = 0$;
- 4: **if** ($P^s = \emptyset$) **then**
- 5: return;
- 6: **end if**
- 7: **while** ($|P^s| > 0$ && $i_1 < l_1$) **do**
- 8: randomly take a pods set $\tilde{P}$ from P^s and delete it from P^s ;
- 9: $RP \leftarrow \text{IP}(\tilde{P}, R^a)$;
- 10: **if** ($KS(RP) < \tilde{n}$) **then**
- 11: $\tilde{n} \leftarrow KS(RP)$; $\tilde{RP} \leftarrow RP$;
- 12: **end if**
- 13: $i_1 \leftarrow i_1 + 1$;
- 14: **end while**
- 15: **if** ($i_2 < l_2$) **then**
- 16: $i_2 \leftarrow i_2 + 1$;
- 17: goto 1;
- 18: **end if**
- 19: **end if**
- 20: **for** (each $(r, p) \in \tilde{RP}$) **do**
- 21: assign p to w ; assign r to p ; add p to P_w ; remove p from P^a ;
- 22: remove r from R^a ;
- 23: **end for**

Additionally, there are only $|R^a| \cdot |\tilde{P}|$ decision variables and constraints in IP, which can be rapidly solved using commercial software. In summary, the complexity of LSMM is $o(l_1 \cdot l_2 \cdot |P^a| \cdot |R^a|)$. Furthermore, the complexity of the PS&TA approach is $o(|O| \cdot |P^a|(|P^b| + |R^a|))$.

4. Numerical experiments

All instances were executed on an x64 PC (Intel Core i7-8650, 3.60 GHz, and 32.0 GB RAM). The experimental framework was developed using C# 8.0, and Gurobi 9.5.2 was employed to solve mathematical models. Next, we introduce the experimental framework, performance evaluation, sensitivity analysis, and some managerial insights in Sections 4.1, 4.2, 4.3, and 4.4, respectively.

4.1. Experimental framework

RAWSim-O is a simulation framework for RMFS, detailed in Section 2.2. Numerous studies have recently applied RAWSim-O to evaluate the performance of related algorithms. RAWSim-O was utilized by Merschformann et al. [7] to examine the effect of various rule combinations on diverse performance metrics, including distance traveled per robot, order throughput, order turnaround time, and pile-on. Xie et al. [8] introduced order split into RMFS and applied RAWSim-O for evaluation. Jiao et al. [28] conducted online joint optimization for OA

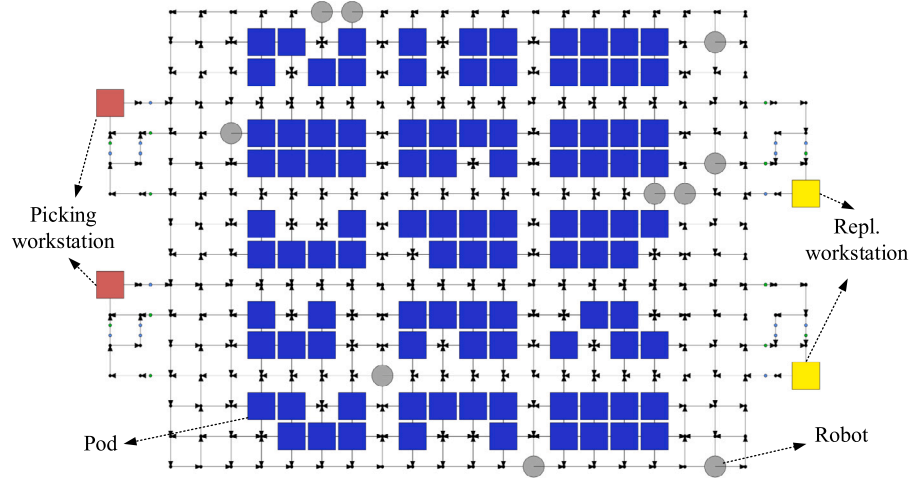


Fig. 5. Layout of small-scale instances. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

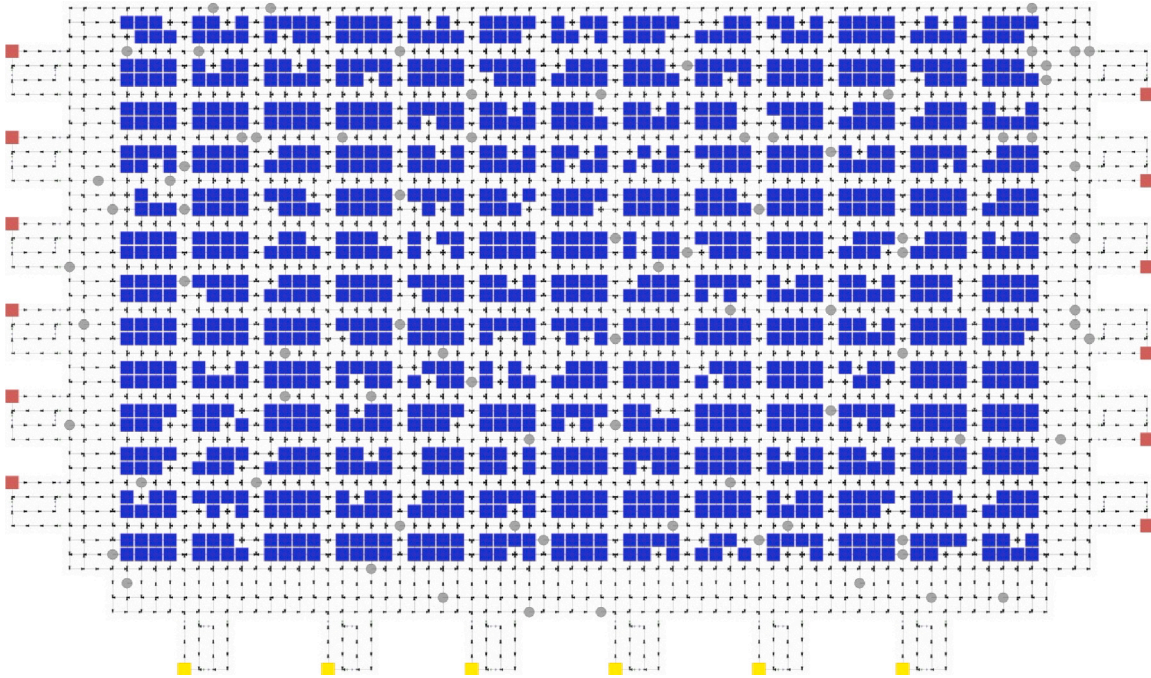


Fig. 6. Layout of large-scale instances. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

and PS in RMFS and utilized the extended RAWSim-O to validate their proposed model and methodology.

Moreover, this study developed a new solver in the extended EE-RAWSim-O to solve OJOOPT. Specifically, the new solver includes two options: utilizing Gurobi to solve M1 (referred to as M1G) and HADGS. Whenever a c_w is not less than δ , the new solver is triggered. For more information regarding the EE-RAWSim-O, please refer to the source code at <https://github.com/LBJiao/EE-RAWSim-O>.

In actual RMFS, robots charge whenever their battery is low; moreover, they do not always remain in working mode. However, as this study primarily focuses on decision-making issues in RMFS at the operational level, the robot charging issue in simulation is considered outside the scope of this work; thus, all robots' battery levels in RAWSim-O are considered unlimited. RAWSim-O input primarily consists of three parts: layout, instance, and solver configuration.

4.1.1. Layout configuration

To enhance the evaluation of both the model and approach, problem instances are categorized into small- and large-scale instances. The

specific layout parameters, including the quantity of pods, storage locations, picking and replenishment workstations, picking workstation capacity, and length of queuing zone, are presented in Table 5. The top view of the layout for small-scale instances is shown in Fig. 5, and that for large-scale instances is shown in Fig. 6. In Figs. 5 and 6, a blue square denotes a pod, a gray circle denotes a robot, a red square denotes a picking workstation, and a yellow square denotes a replenishment workstation.

4.1.2. Instance configuration

Table 6 provides values for some parameters used to generate different instances, including the number of orders in backlog, robots, workstations, and capacity threshold. Instance configuration also includes information such as order and inventory generation, robot movement, and workstation operations. We set the aforementioned parameters based on the study reported in [7], as detailed in Appendix A. Notably, the number of lines and items per order are created from truncated normal distributions, which are intended to simulate the characteristics of

Table 5
Layout parameters.

Parameter	Value	
	Small-scale	Large-scale
Quantity of pods	100	1200
Quantity of storage locations	120	1352
Quantity of picking workstations	2	12
Quantity of repl. workstations	2	6
Picking workstation capacity	6	12
Length of queuing zone	9	15

Table 6
Parameter information for different instances.

Symbol	Description	Value	
		Small-scale	Large-scale
$ O $	Quantity of orders in backlog	70,85,100	300,400,500
δ	Capacity threshold	1,2,3,4	1
$ R $	Quantity of robots	6,7,8,9,10,11,12	60,70,80,90,100,110,120
$ W $	Quantity of workstations	2	4,6,8,10,12

Table 7
Decision-making approaches in the simulation experiment.

Problem	SEQU	OJOOP	OJOOPT
OA	Pod-Match		
PS	Demand	M2G	M1G, HADGS
TA	Balanced	Balanced	
PR	Nearest	Nearest	Nearest
PP	WHCA _n [*]	WHCA _n [*]	WHCA _n [*]
ROA	Pod-Batch	Pod-Batch	Pod-Batch
RPS	Emptiest	Emptiest	Emptiest
RTA	Balanced	Balanced	Balanced

online retail orders. In addition, to generating inventory, the type and size of SKUs follow exponential and uniform distributions, respectively.

4.1.3. Solver configuration

In the simulation experiment, the sequential decision approach used in practice (SEQU) and the approach for solving the MILP model utilizing Gurobi (M2G) proposed by Jiao et al. [28] in the online joint optimization of OA and PS (OJOOP) are compared with the proposed approach. We conduct small-scale instances for M1G, M2G, HADGS, and SEQU. M1G is a method that utilizes Gurobi to solve mathematical model M1, which is an exact method that serves as a benchmark for heuristic methods. Owing to their inability to solve large-scale instances within a finite time frame, both M1G and M2G are unsuitable for such scenarios. For example, even after running for 24 h at $t = 1$ in a large-scale instance, M1G still leaves a gap of 15.4%. Consequently, we exclusively evaluated HADGS and SEQU in large-scale instances.

The relevant decision-making problems involved in RMFS at the operational level were defined in Section 1.1. Their corresponding decision-making approach used in the simulation experiments are presented in Table 7. The decision-making approach used in SEQU is a combination of Pod Match, Demand, Balanced, Nearest, WHCA_n^{*}, Pod Batch, Emptiest, and Balanced. The choice of these aforementioned approaches in this study is based on the experimental findings reported by Merschformann et al. [7], because the combination of approaches ultimately achieves the highest throughput. See [7] for further details of such approaches, which are not repeated in this paper. The remaining decisions in OJOOPT and OJOOP are the same as SEQU except for the approaches for their respective joint decisions.

We set the threshold τ to 30 min and the value of α_3 to 1000. In addition, the values of some important parameters in M1G and HADGS are determined by a series of preliminary experiments. The two weight coefficients of the optimization objectives are set to $\alpha_1 = -40$ and $\alpha_2 = 1$, respectively. We set the pod set quantity of evaluations to

$l_1 = 10$, and the quantity of local searches to $l_2 = 3$. We also set the threshold of available capacity of the workstation that triggers OJOOPT to $\delta = 1$.

All results obtained by the model-based method (including M1G and M2G) in the experimental results are optimal solutions; that is, there is no optimal gap.

4.1.4. Performance metrics

This study employs the following four performance metrics to evaluate the relevant approaches:

- Throughput (TP), which represents the operational efficiency of RMFS, is calculated by dividing the aggregate quantity of orders fulfilled by the hours spent fulfilling them.
- Robot distance (RD), which represents the aggregate distance traveled by all robots, is calculated by summing up the summation of distances traveled by each robot within the simulation horizon. A smaller RD indicates lower RMFS power consumption and running costs.
- Order distance (OD), which represents the average distance traveled by robots per order, is calculated by dividing RD by the aggregate quantity of orders fulfilled. Similar to the RD , the smaller the OD , the less robot power consumption per order on average, and the lower the RMFS running cost.
- Pile-on (PO), which represents the average quantity of orders fulfilled per pod movement, is calculated by dividing the aggregate quantity of fulfilled orders by the aggregate quantity of pod movements. The larger the PO , the more orders are fulfilled per pod movement, which improves RMFS operational efficiency.

To mitigate the impact of the dynamic environment on RMFS, each instance is run ten times for a simulation horizon of 2 h.

4.2. Performance evaluation

4.2.1. Small-scale instances

Table 8 presents the performance evaluation results for small-scale instances ($|R| = 10$ and $|O| = 70, 85, 100$). Column “ V ” represents the performance metric value. Column “ ΔV ” (in %) represents the gap between the relevant approaches and M1G regarding V . Column “AT” (denoted in seconds) represents the average computation time of the approach and is calculated by dividing the aggregate calculation time of the approach in the simulation horizon by the overall number of calls.

Table 8 shows that, for performance metrics TP , RD , and OD in small instances, M1G is significantly better than SEQU and M2G under different $|O|$ conditions. However, for performance metric PO , M1G is better than SEQU, albeit worse than M2G. Specifically, for the TP , RD , OD , and PO , the average ΔV of SEQU is 12.41, 22.05, 39.39, and 34.49%, respectively; the average ΔV of M2G is 1.22, 3.57, 4.58, and -4.43%, respectively. Furthermore, the ΔV of HADGS is minimal, with average values of 0.22, 1.16, 1.55, and 2.02% for the four performance metrics, respectively.

Note that, for PO , the ΔV of M2G is negative; thus, the PO of M1G is smaller than that of M2G. The primary reason for the aforementioned results is that the optimization objective of M2G is to minimize pod movements, whereas that of M1G is to minimize the robot travel distance while maximizing the quantity of orders fulfilled by selected pods. Therefore, when M1G achieves the optimal objective value, it does not necessarily result in an optimal quantity of pod movements. Consequently, the quantity of pod movements in M2G is lower than in M1G for fulfilling the same quantity of orders. Consequently, the PO of M1G is smaller than that of M2G.

Table 8 shows that in small-scale instances, M1G’s AT is less than that of M2G, whereas the AT of SEQU and HADGS, less than 0.00 s, is almost insignificant. As the quantity of orders increases, the AT for

Table 8
Calculation results of small-scale instances.

O	Metrics	M1G		HADGS			SEQU			M2G		
		V	AT	V	ΔV	AT	V	ΔV	AT	V	ΔV	AT
70	TP	310.30		309.95	0.11		275.20	11.31		306.67	1.17	
	RD	15139	0.06	15281	0.94		17961	18.64		15422	1.87	
	OD	24.39		24.65	1.07	0.00	32.63	33.78	0.00	25.14	3.08	0.25
	PO	2.98		2.91	2.35		2.02	32.21		3.08	-3.36	
85	TP	313.60		312.90	0.22		275.78	12.06		309.80	1.21	
	RD	14404	0.09	14593	1.31		17898	24.26		14994	4.1	
	OD	22.97		23.28	1.35	0.00	32.45	41.27	0.00	24.12	5.01	0.33
	PO	3.18		3.12	1.89		2.09	34.28		3.38	-6.29	
100	TP	314.50		313.48	0.32		270.92	13.86		310.44	1.29	
	RD	14122	0.12	14295	1.23		17406	23.25		14791	4.74	
	OD	22.45		22.95	2.23	0.00	32.13	43.12	0.00	23.72	5.66	0.39
	PO	3.30		3.24	1.82		2.08	36.97		3.42	-3.64	

Table 9
Calculation results of M1G and HADGS with different parameter combinations.

ID	Parameters			M1G					HADGS					
	O	R	P	TP	RD	OD	PO	AT	TP	RD	OD	PO	Δ	AT
1	100	10	100	314	23325	37.14	3.09	0.12	313	23523	37.64	3.03	1.12	0.00
2	100	20	100	316	24523	38.8	3.29	0.38	314	24795	39.48	3.25	1.18	0.00
3	100	10	200	309	23149	37.46	3.43	0.85	307	23329	38.05	3.37	1.19	0.01
4	100	20	200	314	24365	38.8	3.36	1.13	312	24564	39.37	3.32	1.03	0.01
5	100	30	200	313	25107	40.11	3.41	1.65	312	25378	40.67	3.34	1.11	0.01
6	150	10	100	326	22059	33.83	3.7	0.59	324	22245	34.38	3.64	1.18	0.01
7	150	20	200	321	23820	37.1	3.47	1.29	320	24026	37.6	3.42	0.99	0.01
8	150	30	200	321	25100	39.1	3.45	1.97	319	25151	39.48	3.41	0.74	0.01
9	150	30	300	333	27805	41.75	3.26	2.35	331	28084	42.48	3.21	1.22	0.02
10	150	40	400	329	25514	38.78	3.35	3.19	327	25702	39.36	3.28	1.24	0.02
11	200	10	100	328	21617	32.96	3.71	1.03	327	21808	33.35	3.67	0.86	0.01
12	200	20	200	327	23912	36.56	3.73	1.44	325	24100	37.08	3.7	0.91	0.02
13	200	30	100	332	25733	38.75	3.44	1.25	331	26007	39.35	3.39	1.09	0.02
14	200	20	300	339	23947	35.32	3.81	2.38	337	24135	35.81	3.75	1.09	0.02
15	200	30	200	326	25096	38.49	3.51	2.13	325	25274	38.88	3.48	0.72	0.02
16	250	40	200	327	25711	39.31	3.54	2.49	326	25866	39.67	3.5	0.74	0.02
17	250	40	300	338	29603	43.79	3.54	3.06	337	29760	44.15	3.49	0.77	0.03
18	250	50	300	337	28696	42.57	3.51	3.36	335	28944	43.2	3.45	1.16	0.03
19	250	40	400	327	26010	39.77	3.15	4.01	326	26163	40.13	3.11	0.77	0.03
20	250	50	400	327	26133	39.96	3.2	4.32	325	26359	40.61	3.17	1.01	0.03

M1G and M2G shows an upward trend, whereas the AT for SEQU and HADGS remains relatively stable. The aforementioned results indicate that using Gurobi to solve MILP models exhibits a greater sensitivity to order quantity fluctuations compared to heuristic approaches.

Additionally, for exact approach like M1G, the computation time is influenced by multiple parameters such as the number of orders, pods, and robots. To determine the size of instances that the exact approach can handle, we must modify the layout of Fig. 5 by adding four vertical aisles and four horizon aisles to arrange more pods. Other parameters, except for the number of orders, pods, and robots, are consistent with the small-scale instances. We ran the test on 20 instances of different parameter combinations, and the results are shown in Table 9. The column “ Δ ” (in %) represents the average gap between HADGS and M1G regarding the four performance metrics.

Table 9 shows that, with the gradual increase of instance scale, the AT of M1G increases rapidly, reaching a maximum of 4.32 s; however, the AT of HADGS increase slowly, with a maximum of less than 0.03 s. In a real RMFS, the decision time is generally required to be no more than 3 s [22]. Furthermore, for instances of different scales, the value of Δ is relatively small, averaging at 1.01%.

4.2.2. Large-scale instances

Table 10 presents the performance evaluation results for large-scale instances ($|R| = 100$ and $|O| = 300, 400, 500$). Column “ ΔV ” (in %) represents the gap between SEQU and HADGS regarding V . Table 10 shows that, in large instances, HADGS significantly outperforms SEQU in terms of all the performance metrics, while its AT is only slightly

Table 10
Calculation results of large-scale instances.

O	Metrics	SEQU			HADGS	
		V	ΔV	AT	V	AT
300	TP	1676.46	7.73		1816.91	
	RD	244610	29.88		188341	
	OD	72.97	40.79	0.09	51.83	0.17
	PO	2.34	34.82		3.59	
400	TP	1646.35	11.10		1852.00	
	RD	238881	30.96		182409	
	OD	72.57	47.35	0.12	49.25	0.18
	PO	2.32	45.67		4.27	
500	TP	1631.90	13.40		1884.33	
	RD	235571	35.23		174206	
	OD	72.18	56.13	0.15	46.23	0.21
	PO	2.34	50.11		4.69	

greater than SEQU’s AT. For example, at $|O| = 100$, HADGS’s AT is only 0.06 s longer than SEQU’s; however, the four performance metrics improved by 13.40, 35.23, 56.13, and 20.11%, respectively. Furthermore, the AT of HADGS exhibits a maximum value of 0.21 s. Thus, the HADGS calculation time, which is specifically designed to address large-scale instances, can meet practical application requirements.

Additionally, we further compare the proposed online approach (i.e., HADGS) with the offline approach (i.e., bi-level memetic (BLM)) proposed by Teck and Dewil [25]. Because HADGS is an online approach, it must be run in EE-RAWSim-O, while BLM is an offline

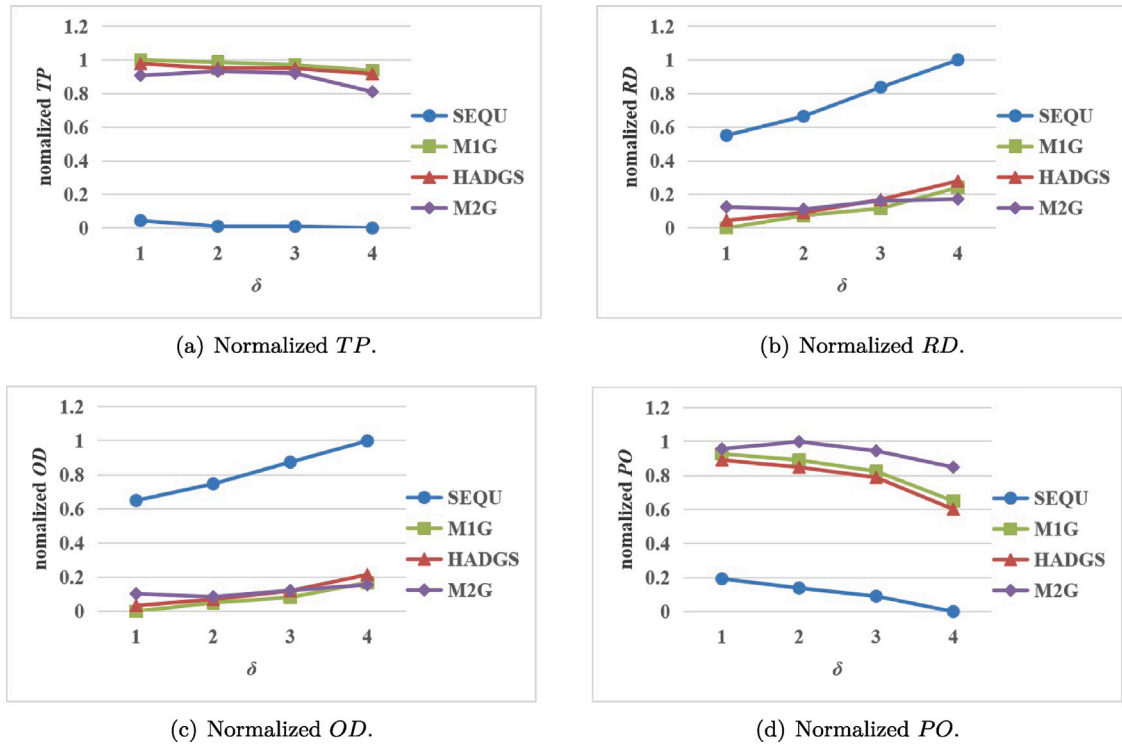
Fig. 7. Influence of parameter δ .

Table 11

Calculation results of HADGS and BLM in large-scale instances.

ID	HADGS					BLM		
	MS	$\Delta 1$	RD	$\Delta 2$	AT	MS	RD	AT
1	1647	11.43	195	13.37	0.11	1478	172	11 744
2	1632	10.2	200	12.99	0.12	1481	177	12 187
3	1643	10.19	202	12.22	0.10	1491	180	11 828
4	1685	11.74	197	13.22	0.13	1508	174	11 683
5	1663	10.72	196	13.95	0.12	1502	172	11 792
A	1654	10.86	198	13.15	0.12	1492	175	11 847

approach and can be run directly. Therefore, to compare HADGS with BLM, corresponding modifications must be made to EE-RAWSim-O and BLM, as detailed in Appendix B. We use makespan (MS) and robot distance (RD) as performance metrics. The test is conducted on a total of five instances, with each instance running ten times, and the results are averaged. The results are shown in Table 11, where column “ $\Delta 1$ ” (in %) represents the gap between HADGS and BLM regarding MS, column “ $\Delta 2$ ” (in %) represents the gap between HADGS and BLM regarding RD, and row “A” represents the related values average.

As shown in Table 11, compared to HADGS, although BLM reduced by an average of 10.86 and 13.15% in MS and RD respectively, it significantly increased AT to 11 847 s. Although the offline approach performed better, additional computational effort was required in which further cost was accrued. Obviously, such a significant AT is unacceptable for any real-time decision-making system.

4.3. Sensitivity analysis

4.3.1. Capacity threshold of workstations

As stated in Section 2.1, the δ parameter represents the threshold for the workstation available capacity. The solver is triggered when any workstation's available capacity exceeds or equals δ . This subsection explores the influence of the δ value on the performance metrics for small-scale instances. The larger the δ , the greater the quantity of orders assigned in a single decision. This leads to a stronger positive impact on

performance metrics owing to the increased correlation between these orders. However, a larger δ results in longer time intervals between consecutive decisions, potentially retracting some pods that can still fulfill orders in $|O|$ back to the storage zone, thereby adversely impacting performance metrics. Therefore, the value of δ significantly impacts the performance of associated decision approaches. The influence of different δ values in small-scale instances ($|R| = 10$ and $|O| = 100$) is illustrated in Fig. 7.

As observed in Fig. 7, for all performance metrics, the performance of SEQU, HADGS, and M1G deteriorates with increasing δ , whereas the performance of M2G initially improves and subsequently deteriorates. SEQU, HADGS, and M1G demonstrate peak performance when $\delta = 1$, whereas M2G demonstrates peak performance when $\delta = 2$. There are several reasons for these aforementioned disparities. First, the correlation between orders is not considered in SEQU and thus cannot positively impact performance. Therefore, the greater the value of δ , the greater the negative effect and the worse the performance of SEQU. Second, the correlation between orders is well considered in M2G, which can positively impact performance. Therefore, with the increase in δ , the negative effect gradually outweighs the positive effect, resulting in an initial improvement and subsequent deterioration in M2G performance. Third, HADGS and M1G are joint decision-making methods that encompass OA, PS, and TA. These methods are effective only when the quantity of idle robots is not less than the quantity of pods that must be transported. Although HADGS and M1G also consider the correlation between orders, it can positively impact performance. However, assigning more orders means that more pods must be transported, which may negatively impact performance. For HADGS and M1G, the increase in δ exhibits a greater negative impact on their performance than a positive impact, leading to a decline in their performance. In summary, in the simulation experiment, we set δ to 1 for SEQU, HSDGS, and M1G, while δ is set to 2 for M2G.

4.3.2. Quantity of robots

The scale of the robot fleet deployed in RMFS, that is, the quantity of robots, significantly impacts RMFS operational efficiency. The appropriate quantity of robots can enhance productivity, improve order

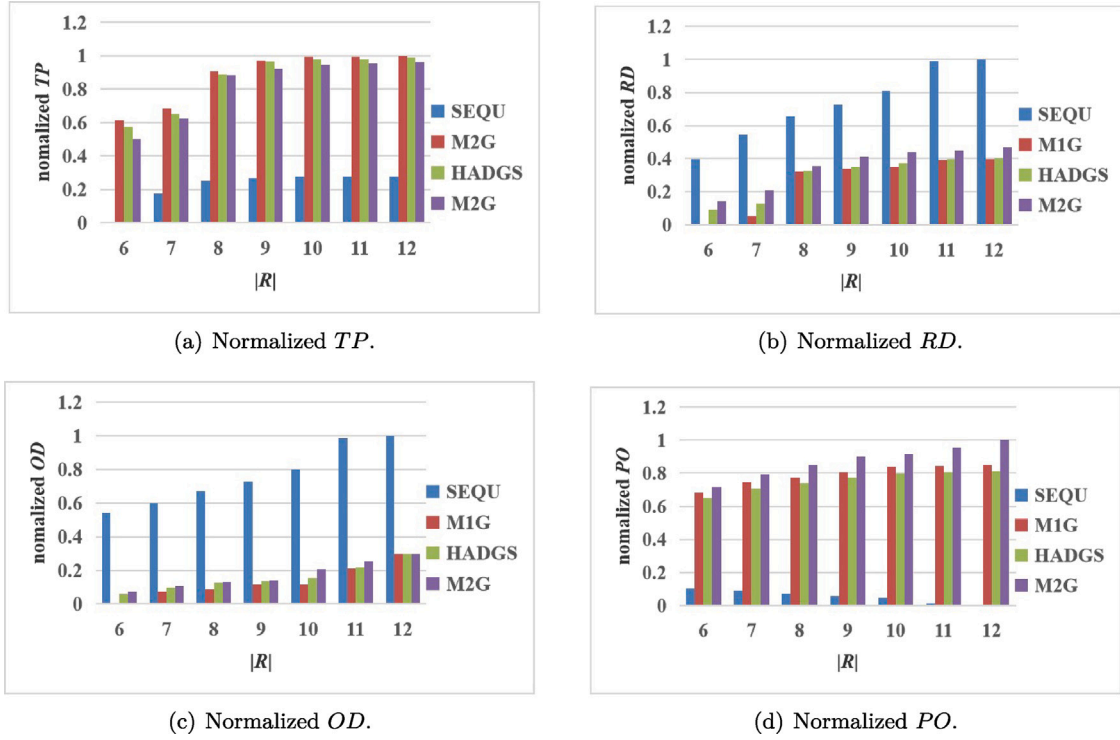


Fig. 8. Influence of different quantities of robots in small-scale instances.

fulfillment speed, and reduce errors in picking and sorting operations. When the quantity of robots deployed in the RMFS is sufficient, all transportation tasks can be immediately executed by robots. This avoids the situation of “picker waiting parts” insofar as possible. Therefore, increasing the quantity of robots deployed in RMFS can decrease the waiting time for pickers and enhance resource utilization, which positively impacts operational efficiency. Conversely, an increase in the quantity of robots results in longer queuing times in the queuing zone and amplifies the likelihood of congestion, collisions, and deadlocks along robots paths, thereby negatively impacting operational efficiency. Therefore, we conducted experiments to examine the impact of varying the quantity of robots on pertinent performance metrics.

Fig. 8 displays the influence of different quantities of robots in small-scale instances ($|R| = 6, 7, 8, 9, 10, 11, 12$ and $|O| = 100$). The TP of all four approaches demonstrates an upward trend as the quantity of robots increases, eventually stabilizing after reaching a threshold of ten robots. When the quantity of robots is small, there are more transportation tasks than available robots, leading to causes of “picker waiting parts”. In this scenario, the positive impact of increasing the quantity of robots outweighs the negative impact, resulting in improved TP as the robot quantity increases. When the robot quantity reaches saturation, where there are consistently more available robots than transportation tasks, leading to causes of “parts waiting picker”. In this scenario, the positive impact initially generated by increasing the quantity of robots is gradually counterbalanced by the negative impact, resulting in a plateau of TP growth. For the RD , an increase in robot quantity involved in transportation inevitably increases the aggregate distance traveled by all robots. Additionally, because the quantity of orders fulfilled in a transportation task is typically greater than 1 (i.e., $PO > 1$), the OD also increases with the increase in robot quantity. For the PO , as the quantity of robots increases, HADGS, M1G, and M2G all significantly increase in contrast to SEQU, which decreases. This discrepancy can be attributed to the fact that the increase in TP for HADGS, M1G, and M2G is more pronounced compared to that for SEQU as shown in Fig. 8(a). Moreover, an increase in TP is indicative of a higher quantity of orders that require fulfillment, while an increase

in the quantity of robots implies a higher quantity of pod movements that can be provided. Therefore, per the definition of PO , when the increase in TP exceeds that of the robots, PO will increase, as observed in HADGS, M1G, and M2G. Conversely, when the increase in TP is less than that of the robots, PO will decrease, as observed in SEQU.

Fig. 9 shows the influence of different quantities of robots in large-scale instances ($|R| = 60, 70, 80, 90, 100, 110, 120$, and $|O| = 500$). As the quantity of robots increases, the changing trend of SEQU and HADGS in large-scale instances remains consistent with that in small-scale instances across the four performance metrics. The reasons for these changes are also the same as those observed in small-scale instances. The only disparity is that, as the quantity of robots increases, the PO of HADGS exhibits an upward trend in large-scale instances, albeit a downward trend in small-scale instances. The reason for this disparity is that in large-scale instances, the increase in the quantity of robots surpasses the corresponding increase in TP for HADGS. Consequently, the PO of HADGS decreases with the increasing robot quantity.

The aforementioned results demonstrate that, by optimizing relevant decision-making approaches, the quantity of robots deployed in RMFS could be effectively decreased. This suggests that enhancing the performance of the decision-making approach can lead to more optimal resource allocation and utilization, resulting in a reduction of the necessity for numerous robots. Specifically, in small-scale instances, the TP of M1G when $|R| = 6$ remains 8% higher than that of SEQU when $|R| = 12$ while reducing the corresponding RD by 38%; in large-scale instances, the TP of HADGS when $|R| = 60$ remains 6% higher than that of SEQU when $|R| = 120$ while reducing the corresponding RD by 50%. Hence, M1G and HADGS employ at most half the quantity of robots employed in SEQU to achieve an equivalent throughput for both small and large instances. Meanwhile, the corresponding distance traveled by robots is only 62 and 57% of that in SEQU, respectively.

4.3.3. Quantity of workstations

Based on a survey of an RMFS owned by a large retail company, we observed that managers frequently adjust the quantity of workstations (referring to active workstations) based on the total volume of customer

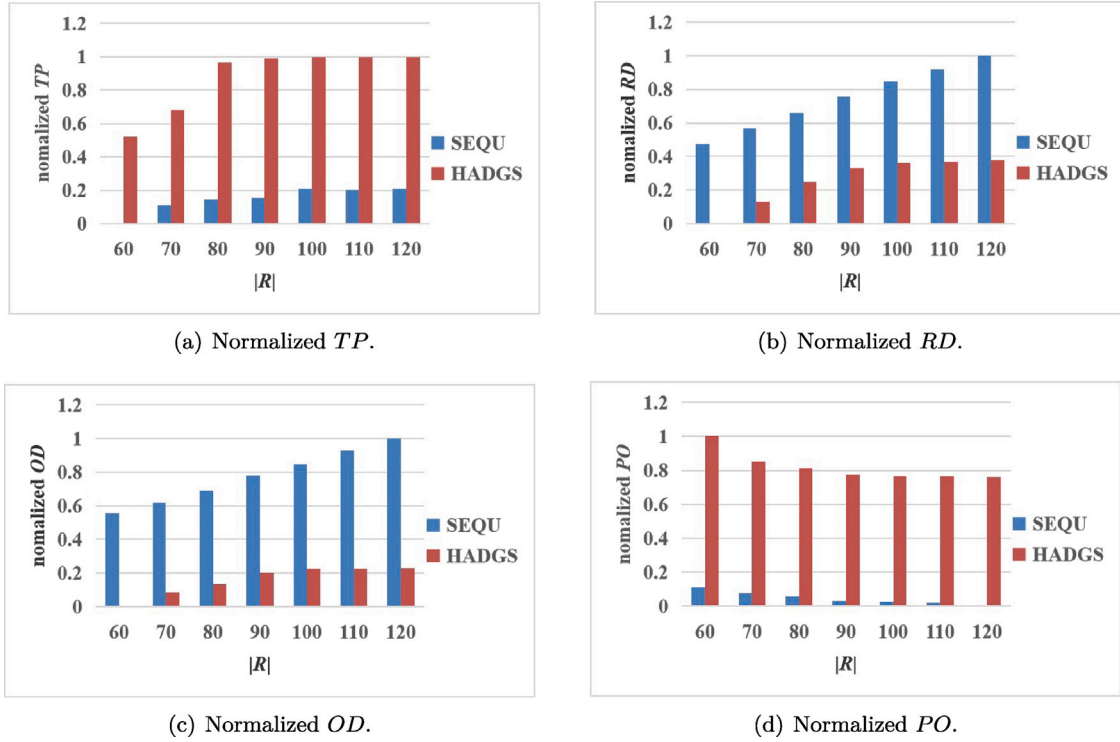


Fig. 9. Influence of different quantities of robots in large-scale instances.

orders. This proactive adjustment helps to minimize operating costs, optimize resource allocation, and enhance efficiency. Specifically, managers activate some workstations during the peak season and close some workstations during the off-season. Therefore, next, we investigate the influence of the quantity of workstations on an RMFS. Fig. 10 illustrates the influence of different quantities of workstations in large-scale instances ($|R| = 100$ and $|O| = 500$), where DV represents the gap in performance metric values between HADGS and SEQU.

As observed in Fig. 10, HADGS outperforms SEQU in all performance metrics for different workstation quantities. With the increase in the quantity of workstations, TP and RD of HADGS and SEQU increased gradually, OD of HADGS and SEQU decreased gradually, PO of SEQU increased gradually, PO of HADGS increased initially and subsequently decreased. The aforementioned results indicate that when the quantity of orders from customers is small (i.e., expected TP is small), the RD could be reduced by reducing the quantity of workstations. Therefore, workstation quantity adjustment based on the order volume can optimize cost in situations where the expected TP is relatively low.

In addition, as the workstation quantity increases, the DV for TP , RD , and PO initially increases and subsequently decreases, while the DV for OD gradually decreases. OD is equivalent to the throughput divided by the distance traveled by robots. Therefore, as the workstation quantity increases, OD decreases when the increment of the TP is less than that of the RD . The aforementioned results show that the performance improvement of HADGS over SEQU increases initially and subsequently decreases as the quantity of workstations increases. Moreover, as illustrated in Fig. 10(a), the TP of HADGS at $|W| = 10$ is almost the same as that of SEQU at $|W| = 12$, indicating that HADGS requires fewer workstations when the same TP is reached.

4.4. Managerial insights

Based on the results of these simulation experiments, the following warehouse managerial insights were identified:

- Reducing the number of pod movements does not necessarily enhance warehouse operational efficiency, which deviates from conventional perspectives in existing literature. For example, from the perspective of system operational costs, Teck and Dewil [26] found that only minimizing the number of pod movements is insufficient and using distance as the optimization objective is more suitable for optimizing operational costs. From the perspective of system operational efficiency (i.e. throughput), this study found no correlation between the number of pod movements and system operational efficiency, that is, a reduction in the number of pod movements may not necessarily improve system operational efficiency. Specifically, compared with M1G, although M2G increased by 4.43% on PO , it decreased TP by 1.22%.
- OJOOPT can enhance warehouse operational efficiency and reduce its operational costs. The experimental results in Table 10 show that HADGS demonstrated a 10.75% increase in TP and a 32.02% decrease in RD in large-scale instances compared to a state-of-the-art sequential approach.
- OJOOPT can achieve the same throughput with fewer resources (workstations). Warehouse managers have the flexibility to adjust the number of workstations deployed based on customer orders, further reducing warehouse operational costs. The experimental results in Fig. 10(a) demonstrated that in large-scale instances, the number of workstations required for HADGS to achieve the same TP was only 83% of SEQU.

5. Conclusion

This study investigated the online joint optimization for OA, PS, and TA in RMFS during the order picking process. To enhance decision-making efficiency, the study accounted for the impact of current decisions on future decisions. A dynamic system model for RMFS was established, alongside the development of a novel MILP model to address critical challenges. To solve large-scale instances effectively, a customized HADGS was devised.

The proposed model and approach were rigorously evaluated using small- and large-scale instances within established dynamic systems.

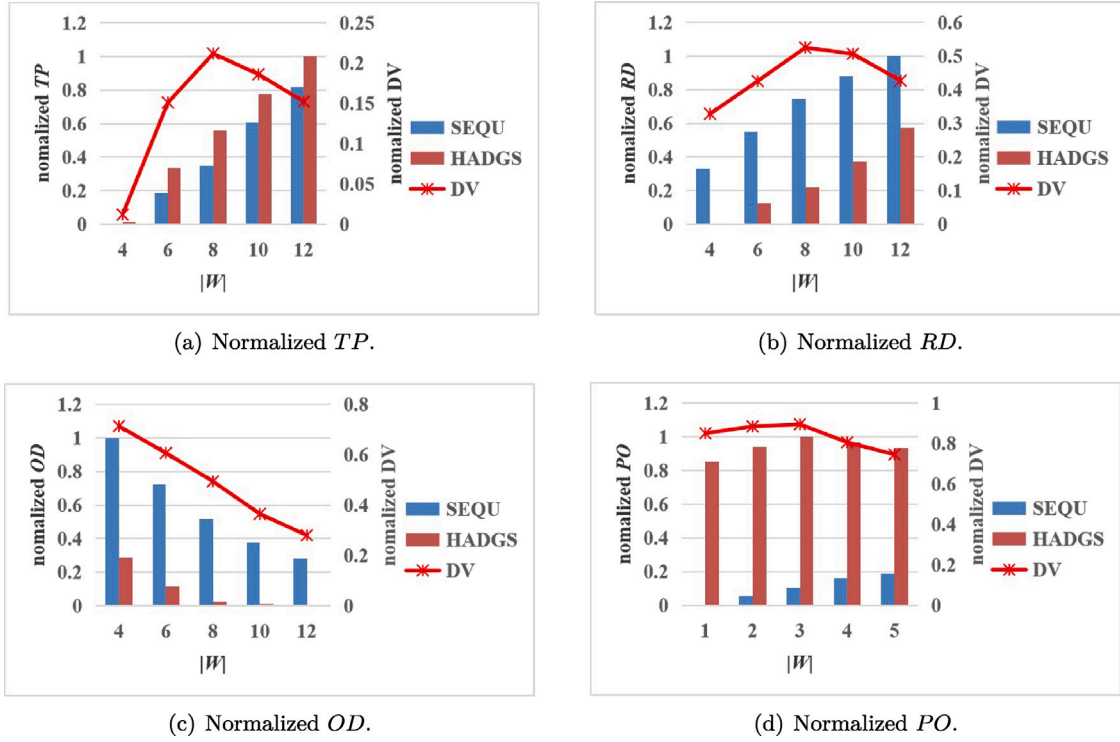


Fig. 10. Influence of quantity of workstations.

Computational results revealed that, in small-scale instances, the proposed M1G outperformed M2G by improving TP , RD , and OD by 1.22, 3.57, and 4.58%, respectively, although it reduced the performance of PO by 4.43%. In large-scale instances, compared to SEQU, the proposed HADGS improved the performance of TP , RD , OD , and PO by 10.75, 32.02, 48.09, and 43.61%, respectively. Furthermore, HADGS required only 50% of the robots and 83% of the workstations used by SEQU to achieve comparable throughput, indicating superior efficiency and resource utilization in optimizing the RMFS order-picking process.

Future research can explore proactive-reactive scheduling frameworks, which combine the predictive strengths of proactive scheduling with the adaptability of reactive scheduling. By leveraging machine learning algorithms and real-time simulation data, these frameworks could significantly enhance resource allocation and task scheduling in RMFS, addressing the challenges of online optimization problems. Combining machine learning predictions with data generated from real-time simulation can enhance the computational efficiency of real-time decision-making processes [51]. Offline simulation learning offers valuable insights for addressing online decision-making challenges. However, its accuracy for offline RMFS prediction requires further validation, positioning it as a focal point for future research. Additionally, the Optimal Computing Budget Allocation (OCBA) algorithm [52,53] has the potential to reduce simulation model runtime by selecting the optimal parameter combinations and employing confidence interval methods to identify the best parameter configurations. Consequently, utilizing simulation optimization methods such as OCBA to test various combinations of optimization methods is a promising direction for future research.

CRedit authorship contribution statement

Guoshuai Jiao: Writing – original draft, Software, Methodology, Formal analysis. **Min Huang:** Supervision, Project administration, Conceptualization. **Yang Song:** Supervision, Software, Conceptualization. **Haobin Li:** Visualization, Supervision, Methodology. **Xingwei Wang:** Software, Project administration, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Key R&D Program of China (Grant No. 2021YFB3300900); the National Natural Science Foundation of China Key Supported Project of the Major Research Plan (Grant No. 92267206); the National Natural Science Foundation of China (Grant No. 62032013 and No. 72071183); the Natural Science Foundation of Shanxi Province, China (202203021211194); the Taiyuan University of Science and Technology Scientific Research Initial Funding (W20242023). The authors also gratefully acknowledge the helpful comments and suggestions from the anonymous reviewers, which significantly helped to improve this paper.

Appendix A. Parameter information for the instance configuration

Details of certain parameters for the instance configuration are presented in Table 6, whereas Table A.1 includes additional parameter information pertaining to inventory, order, workstation, and robot.

Appendix B. Modifications to EE-RAWSim-O and BLM

To compare HADGS with BLM, EE-RAWSim-O and BLM must be modified accordingly:

(1) Modification to EE-RAWSim-O:

- Change the pod repositioning (i.e., PR) decision from Nearest to Fixed, that is, the storage position of the pod is fixed.
- To avoid triggering replenishment operations, we set the initial inventory to 0.95.

Table A.1
Some parameters' information for the instance configuration.

	Parameter	Value
Inventory	Initial inventory	70%
	Storage space for a pod	100 slots 500 slots
	SKU size	Uniform distribution between 2 and 8
	Quantity of SKU types	100 1000
	SKU popularity/frequency	Exponential distribution: $\lambda = 0.5$
Order	Quantity of order lines in the picking order	Truncated normal distributions: $\mu = 1, \sigma = 1, \min = 1, \max = 5$
	Quantity of items in the order line	Truncated normal distributions: $\mu = 1, \sigma = 1, \min = 1, \max = 5$
	Size of backlog of the repl. order	200
	Quantity of items in the repl. order	Uniform distribution between 4 and 12
	ro_1	65%
Workstation	ro_2	85%
	Time to pick an item	3 s
	Time to handle an item at picking workstation	10 s
	Time to put an item on a pod	10 s
	Repl. workstation's capacity	1000 slots
Robot	Acceleration/deceleration	1 m/s ²
	Maximum velocity	1.5 m/s
	Turn time	2.5 s
	Time for lifting and lowering the pod	2.2 s

- To avoid a large number of idle robots at initialization, we set the number of robots to 30.
- Cease operation after the system has finished picking 100 orders and save these orders as a static backlog to be optimized by BLM.

The remaining parameters in EE-RAWSim-O are set the same as those in the large-scale instance.

(2) Modification to BLM:

- One order line may contain multiple units. Therefore, when selecting pods, the pods must meet the order line and multiple units in the order line.
- Inter-station scheduling of pods is not considered, i.e., the robot must first transport the pod back to the storage area before it can be transported to another workstation.
- To visually compare calculation time, the one hour stop criterion is removed.

The warehouse layout and robot specifications required for BLM are consistent with those in EE-RAWSim-O, where $|P| = 1200$, $|O| = 100$, $|R| = 30$, and $|W| = 12$. The settings of other parameters involved in running BLM are the same as those in [25].

Data availability

I have shared the link to my data/code at the manuscript.

References

- [1] Schiffer M, Boysen N, Klein PS, Laporte G, Pavone M. Optimal picking policies in e-commerce warehouses. *Manag Sci* 2022;68(10):7497–517.
- [2] Boysen N, de Koster R. 50 years of warehousing research—An operations research perspective. *European J Oper Res* 2025;320(3):449–64.
- [3] Boysen N, De Koster R, Weidinger F. Warehousing in the e-commerce era: A survey. *European J Oper Res* 2019;277(2):396–411.
- [4] Polten L, Emde S. Scheduling automated guided vehicles in very narrow aisle warehouses. *Omega* 2021;99:102204.
- [5] Wurman PR, D'Andrea R, Mountz M. Coordinating hundreds of cooperative, autonomous vehicles in warehouses. *AI Mag* 2008;29(1):9.
- [6] Azadeh K, De Koster R, Roy D. Robotized and automated warehouse systems: Review and recent developments. *Transp Sci* 2019;53(4):917–45.
- [7] Merschformann M, Lamballais T, De Koster M, Suhl L. Decision rules for robotic mobile fulfillment systems. *Oper Res Perspect* 2019;6:100128.
- [8] Xie L, Thieme N, Krenzler R, Li H. Introducing split orders and optimizing operational policies in robotic mobile fulfillment systems. *European J Oper Res* 2021;288(1):80–97.
- [9] Jaghbeer Y, Hanson R, Johansson MI. Automated order picking systems and the links between design and performance: a systematic literature review. *Int J Prod Res* 2020;58(15):4489–505.
- [10] da Costa Barros ÍR, Nascimento TP. Robotic mobile fulfillment systems: A survey on recent developments and research opportunities. *Robot Auton Syst* 2021;137:103729.
- [11] Lamballais T, Roy D, De Koster RB. Estimating performance in a robotic mobile fulfillment system. *European J Oper Res* 2017;256(3):976–90.
- [12] Xiang X, Liu C, Miao L. Storage assignment and order batching problem in Kiva mobile fulfillment system. *Eng Optim* 2018;50(11):1941–62.
- [13] Weidinger F, Boysen N. Scattered storage: How to distribute stock keeping units all around a mixed-shelves warehouse. *Transp Sci* 2018;52(6):1412–27.
- [14] Yuan R, Graves SC, Cezik T. Velocity-Based storage assignment in Semi-Automated storage systems. *Prod Oper Manage* 2019;28(2):354–73.
- [15] Kim HJ, Pais C, Shen ZJM. Item assignment problem in a robotic mobile fulfillment system. *IEEE Trans Autom Sci Eng* 2020;17(4):1854–67.
- [16] Li X, Hua G, Huang A, Sheu J-B, Cheng T, Huang F. Storage assignment policy with awareness of energy consumption in the Kiva mobile fulfillment system. *Transp Res E: Logist Transp Rev* 2020;144:102158.
- [17] Jiang M, Leung K, Lyu Z, Huang GQ. Picking-replenishment synchronization for robotic forward-reserve warehouses. *Transp Res E: Logist Transp Rev* 2020;144:102138.
- [18] Zhang J, Zhang N, Tian L, Zhou Z, Wang P. Robots' picking efficiency and pickers' energy expenditure: the item storage assignment policy in robotic mobile fulfillment system. *Comput Ind Eng* 2023;176:108918.
- [19] Boysen N, Briskorn D, Emde S. Parts-to-picker based order processing in a rack-moving mobile robots environment. *European J Oper Res* 2017;262(2):550–62.
- [20] Yang X, Hua G, Hu L, Cheng T, Huang A. Joint optimization of order sequencing and rack scheduling in the robotic mobile fulfillment system. *Comput Oper Res* 2021;135:105467.
- [21] Valle CA, Beasley JE. Order allocation, rack allocation and rack sequencing for pickers in a mobile rack environment. *Comput Oper Res* 2021;125:105090.
- [22] Qin H, Xiao J, Ge D, Xin L, Gao J, He S, Hu H, Carlsson JG. JD. com: Operations research algorithms drive intelligent warehouse robots to work. *INFORMS J Appl Anal* 2022;52(1):42–55.
- [23] Zhang S, Zhuge D, Tan Z, Zhen L. Order picking optimization in a robotic mobile fulfillment system. *Expert Syst Appl* 2022;209:118338.
- [24] Zhuang Y, Zhou Y, Yuan Y, Hu X, Hassini E. Order picking optimization with rack-moving mobile robots and multiple workstations. *European J Oper Res* 2022;300(2):527–44.
- [25] Teck S, Dewil R. A bi-level memetic algorithm for the integrated order and vehicle scheduling in a RMFS. *Appl Soft Comput* 2022;121:108770.
- [26] Teck S, Dewil R. Optimization models for scheduling operations in robotic mobile fulfillment systems. *Appl Math Model* 2022;111:270–87.
- [27] Wang B, Yang X, Qi M. Order and rack sequencing in a robotic mobile fulfillment system with multiple picking stations. *Flex Serv Manuf J* 2023;35:509–47.
- [28] Jiao G, Li H, Huang M. Online joint optimization of pick order assignment and pick pod selection in robotic mobile fulfillment systems. *Comput Ind Eng* 2023;175:108856.
- [29] Justkowiak JE, Pesch E. A column generation driven heuristic for order-scheduling and rack-sequencing in robotic mobile fulfillment systems. *Omega* 2023;120:102897.
- [30] Justkowiak JE, Pesch E. Stronger mixed-integer programming-formulations for order-and rack-sequencing in robotic mobile fulfillment systems. *European J Oper Res* 2023;305(3):1063–78.
- [31] Subramanian SP, Chandrasekar SK. Simultaneous allocation and sequencing of orders for robotic mobile fulfillment system using reinforcement learning algorithm. *Expert Syst Appl* 2024;239:122262.

- [32] Zhao Z, Cheng J, Liang J, Liu S, Zhou M, Al-Turki Y. Order picking optimization in smart warehouses with human-robot collaboration. *IEEE Internet Things J* 2024;11(9):16314–24.
- [33] Cheng B, Wang L, Tan Q, Zhou M. A deep reinforcement learning hyper-heuristic to solve order batching problem with mobile robots. *Appl Intell* 2024;54(9):6865–87.
- [34] Yang X, Hua G, Zhang L, Cheng TCE, Choi T-M. Joint optimization of order-and rack-scheduling in KIVA picking systems. *Omega* 2025;135:103286.
- [35] Zou B, Gong Y, Xu X, Yuan Z. Assignment rules in robotic mobile fulfillment systems for online retailers. *Int J Prod Res* 2017;55(20):6175–92.
- [36] Yoshitake H, Kamoshida R, Nagashima Y. New automated guided vehicle system using real-time holonic scheduling for warehouse picking. *IEEE Robot Autom Lett* 2019;4(2):1045–52.
- [37] Gharehgozli A, Zaerpour N. Robot scheduling for pod retrieval in a robotic mobile fulfillment system. *Transp Res E: Logist Transp Rev* 2020;142:102087.
- [38] Li Z, Barenji AV, Jiang J, Zhong RY, Xu G. A mechanism for scheduling multi robot intelligent warehouse system face with dynamic demand. *J Intell Manuf* 2020;31:469–80.
- [39] Li K, Liu T, Kumar PR, Han X. A reinforcement learning-based hyper-heuristic for AGV task assignment and route planning in parts-to-picker warehouses. *Transp Res E: Logist Transp Rev* 2024;185:103518.
- [40] Merschformann M, Xie L, Erdmann D. Multi-agent path finding with kinematic constraints for robotic mobile fulfillment systems. 2017, *ArXiv E-Prints*.
- [41] Lee CK, Lin B, Ng K, Lv Y, Tai W. Smart robotic mobile fulfillment system with dynamic conflict-free strategies considering cyber-physical integration. *Adv Eng Inform* 2019;42:100998.
- [42] Li J, Tinka A, Kiesel S, Durham JW, Kumar TS, Koenig S. Lifelong multi-agent path finding in large-scale warehouses. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 35, 2021, p. 11272–81.
- [43] Sun Y, Zhao N, Lodewijks G. An autonomous vehicle interference-free scheduling approach on bidirectional paths in a robotic mobile fulfillment system. *Expert Syst Appl* 2021;178:114932.
- [44] Luo L, Zhao N, Zhu Y, Sun Y. A* guiding DQN algorithm for automated guided vehicle pathfinding problem of robotic mobile fulfillment systems. *Comput Ind Eng* 2023;178:109112.
- [45] Weidinger F, Boysen N, Briskorn D. Storage assignment with rack-moving mobile robots in KIVA warehouses. *Transp Sci* 2018;52(6):1479–95.
- [46] Roy D, Nigam S, de Koster R, Adan I, Resing J. Robot-storage zone assignment strategies in mobile fulfillment systems. *Transp Res E: Logist Transp Rev* 2019;122:119–42.
- [47] Zhuang Y, Zhou Y, Hassini E, Yuan Y, Hu X. Rack retrieval and repositioning optimization problem in robotic mobile fulfillment systems. *Transp Res E: Logist Transp Rev* 2022;167:102920.
- [48] Boysen N, Schwerdfeger S, Stephan K. A review of synchronization problems in parts-to-picker warehouses. *European J Oper Res* 2023;307(3):1374–90.
- [49] Merschformann M, Xie L, Li H. RAWSim-O: A simulation framework for robotic mobile fulfillment systems. *Logist Res* 2018;11:8.
- [50] Hart PE, Nilsson NJ, Raphael B. A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans Syst Sci Cybern* 1968;4(2):100–7.
- [51] Goodwin T, Xu J, Celik N, Chen CH. Real-time digital twin-based optimization with predictive simulation learning. *J Simul* 2024;18(1):47–64.
- [52] Lee LH, Chen Ch, Chew EP, Li J, Pujowidianto NA, Zhang S. A review of optimal computing budget allocation algorithms for simulation optimization problem. *Int J Oper Res (Taichung)* 2010;7(2):19–31.
- [53] Chen CH, Lee LH. *Stochastic simulation optimization: an optimal computing budget allocation*, vol. 1, World scientific; 2011.